Inferring Beliefs for Search and Rescue from Natural Language

by

Naomi D. Schurr

Submitted to the Department of Aeronautics and Astronautics
in partial fulfillment of the requirements for the degree of

Master of Science in Aeronautics and Astronautics

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

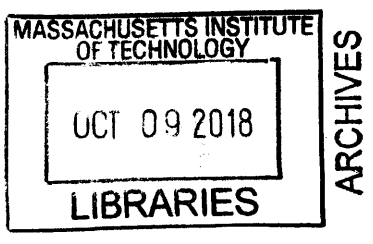
September 2018

© Massachusetts Institute of Technology 2018. All rights reserved.

Author ..... Signature redacted
Department of Aeronautics and Astronautics
August 27, 2018

Certified by ..... Signature redacted
Nicholas Roy
Professor
Thesis Supervisor

Accepted by ..... Signature redacted
Hamsa Balakrishnan
Associate Professor of Aeronautics and Astronautics
Chair, Department Graduate Program Committee



Inferring Beliefs for Search and Rescue from Natural Language

by

Naomi D. Schurr

Submitted to the Department of Aeronautics and Astronautics
on August 27, 2018, in partial fulfillment of the
requirements for the degree of
Master of Science in Aeronautics and Astronautics

Abstract

A learned natural language robotic interface can allow a human operator to intuitively communicate instructions to a robot. A number of models, including probabilistic grounding graphs, have been used to ground natural language input to the real-world tasks a robot must perform. In this thesis, I provide two extensions to existing work in grounding natural language instructions. First, I apply an existing probabilistic grounding graph model in the context of outdoor search and rescue, introducing a new set of groundings to allow a continuous cost map to be inferred from the natural language. Second, I incorporate pool-based active learning into the training of the probabilistic grounding graph model, which shows promise for reducing the number of labeled examples needed to train the model.

Thesis Supervisor: Nicholas Roy
Title: Professor

Acknowledgments

I would first like to thank my thesis adviser Professor Nicholas Roy of the Department of Aeronautics and Astronautics at the Massachusetts Institute of Technology. Through challenges and successes, Nick has been supportive and allowed me to explore a variety of areas.

I would also like to thank the members of the Robust Robotics Group: Dr. Rohan Paul, Dr. Subhro Roy, and Will Vega-Brown were always available for technical discussion and advice; Gregory Stein and Charlie Guan provided considerable encouragement; and all members contributed to the friendly atmosphere of our lab.

Finally, I would like to express my gratitude to my parents, my brother, and my sister for providing me with unfailing support and continuous encouragement throughout my years of study. This accomplishment would not have been possible without them. Thank you.

Contents

1 Introduction **13**

1.1 Grounding natural language to beliefs 14

1.2 Active Learning with Grounding Graphs 17

1.3 Organization 18

2 Related Works **21**

2.1 Grounding Natural Language Instructions 21

2.1.1 Probabilistic Grounding Graphs 23

2.1.2 Other Recent Approaches 23

2.2 Wilderness Search and Rescue 24

2.2.1 Aerial Vehicles in Wilderness Search and Rescue 25

2.2.2 Intelligent Paths for Searching with Micro Aerial Vehicles . . . 25

2.2.3 Interfacing with Search Vehicles 26

2.3 Active Learning 26

2.3.1 Classical Active Learning 27

2.3.2 Active Learning for Natural Language, Grounding, and Structured Prediction 28

3 Probabilistic Grounding Graphs: Formulation and Implementation 31

3.1 Probabilistic Grounding Graph Formulation 31

3.1.1 Conceptual Overview of the Grounding Task 31

3.1.2 Mathematical Formulation of Probabilistic Grounding Graphs 33

3.2 Implementation 37

3.2.1 Process Overview 37
3.2.2 Grammars and the Parse Tree 38
3.2.3 The Factor Graph Representation 40
3.2.4 Groundings 41
3.2.5 Log-Linear Model Features 45
3.2.6 Inference 47
3.2.7 Training 49
3.3 Summary 51

4 Grounding Natural Language to Beliefs for Search and Rescue 53
4.1 Technical Approach Overview 53
4.2 Grounding Natural Language Instructions 55
4.3 Basis function composition 56
4.3.1 The Basis function 56
4.3.2 Composition 57
4.4 Trajectory Generation 58
4.5 Human-Annotated Beliefs 60
4.5.1 Expert-Annotated Training Data 61
4.6 Evaluation and Results 64
4.6.1 Quantitative Results 64
4.6.2 Qualitative Results 71
4.7 Summary 71

5 Active Learning with Grounding Graphs 75
5.1 The Active Learning Algorithm 75
5.2 Experimental Setup 76
5.2.1 Heuristics 77
5.2.2 Data Set 78
5.3 Results 78
5.4 Summary 80

6 Conclusion **83**

A Supplementary Implementation Material **85**

A.1 The Grammar 86

A.2 The Parse Tree 87

A.3 The Factor Graph Representation 88

A.3.1 pyh2sl Groundings 88

A.3.2 H2SL Features 90

A.4 Training Files 92

A.5 Training the Log-Linear Model 94

A.6 H2SL Toy World Instructions 94

List of Figures

1-1 An overview of the first problem addressed in this thesis 15

1-2 A parallel sweep pattern 16

3-1 Possible groundings in a toy world 32

3-2 Parse tree and corresponding factor graph 35

3-3 Parse tree for the text "go to the box" 40

3-4 H2SL Groundings 42

3-5 Grounding steps for "go near the box and the desk" 43

3-6 Grounding for "Go to the box to the left of the desk" 44

4-1 Approach Overview 54

4-2 Parse tree and corresponding factor graph 56

4-3 Planning example for a bimodal distribution 60

4-4 Blank camp map given to annotators to produce lost-person scenarios 62

4-5 Data collection interface 63

4-6 Unnormalized probability map 63

4-7 Samples from the corpus 64

4-8 Output and qualitative comparison 65

4-9 Probability of success for search durations T 68

4-10 Performance summary of 127 language commands 68

4-11 Comparison of the probability of success of various approaches 68

4-12 Trajectory Comparison 70

4-13 Pictorial representation of intermediate groundings 72

4-14 Trajectories when search area is not specified by language 72

5-1 Generic Pool-Based Active Learning Algorithm 76
5-2 Active Learning Results 79

A-1 A simple grammar showing production rules 86
A-2 Using a Natural Language Toolkit parser 87
A-3 The Phrase class 88
A-4 The Grounding class 89
A-5 Sample world model in training data 92
A-6 Sample expert-labeled verb phrase 93
A-7 Training the log-linear model 94
A-8 H2SI Toy World Instructions 95

Chapter 1

Introduction

Natural language interfaces provide an intuitive way for humans to interact with robots. Since humans are accustomed to communicating with each other through natural language, this familiarity can lower the training barrier for a human to begin working with a robot. Enabling a robot to understand natural language can also circumvent the need to design manual controls or graphical user interfaces, and can free a robot operator's hands for other tasks. These advantages have motivated a large body of work in natural language understanding for human-robot interaction.

The process of natural language understanding for use in the robotic domain involves taking a language input, interpreting it in the context of the physical world, and, if the input is an instruction, acting in the physical world. In this thesis, the term *grounding* [12] is used for the interpretation step, where meaning is assigned to words or phrases; the items, actions, constraints, or other ideas in the physical world that can be assigned to the words during the grounding process are potential *groundings*.

Grounding natural language to meaning is challenging because the domains of both language and physical interpretations are large. An already-large vocabulary leads to a larger space of instructions, as the words in the vocabulary can be composed into complex sentence structures. The environment in which a real robot acts is continuous, and may contain many elements of interest that could take on any number of configurations. Formulating a direct mapping from each language string to its

meaning in the physical world is thus infeasible.

While a number of approaches exist to make the problem tractable (see Chapter 3), in this work I focus on the probabilistic grounding graph approach, which includes the Generalized Grounding Graph (G^3) [41], the Distributed Correspondence Graph (DCG) [15], and variants [5, 31, 46, 30]. The probabilistic grounding graph approach instantiates a factor graph according to the parse tree of the natural language instruction. The factors tie sub-phrases of the language to symbols representing the groundings, where the probability of a sub-phrase corresponding to a grounding is given by a log-linear model that has been learned from data. Using conditional independence assumptions, grounding can be performed factor by factor as multiple smaller grounding problems, which are combined into a maximum likelihood grounding of the full instruction. Probabilistic grounding graph models have been successfully used in forklift operation [41], navigation [15], and tabletop manipulation applications [31].

1.1 Grounding natural language to beliefs

In the first section of this thesis, I consider the problem of using a micro aerial vehicle to search for a lost hiker in the outdoors. When a person goes missing in the outdoors, a search team determines where to search using a variety of factors, including the tendencies of missing persons and the place and time the hiker was last seen. To coordinate search efforts, humans may communicate using language; for example, the leader of a search party might tell a pilot, "Search the woods west of the lake", which establishes where the hiker is believed to be and where a micro aerial vehicle should fly. Figure 1-1 shows an overview of the task: given a language instruction on where to search, a vehicle should fly an appropriate trajectory for finding a missing person.

Reproducing the ability to parse language commands and generate intelligent search behavior in an autonomous vehicle is challenging. To make use of language, a robot needs to be able to use the words spoken to evaluate the quality of a candidate search trajectory. A common approach in language grounding for robotic contexts is to separate the problem into two distinct steps. Rather than inferring a trajectory



Figure 1-1: *An overview of the first problem addressed in this thesis.* A search and rescue operator gives a natural language command to an autonomous vehicle, which must execute an appropriate trajectory (in red).

directly from the language instruction, a language model is used to infer a planning problem representation, to which a conventional planner is applied to generate a trajectory. While existing language grounding systems have been demonstrated to generate trajectories for navigation, the trajectories for search and rescue are fundamentally different—in navigation, the final destination generally matters more than the details of the route, while in a coverage search, the opposite is true. As such, the representations that have been used for planning navigation-type trajectories are impractical for planning search trajectories.

For illustrative purposes, consider the instruction "fly to the tree". The planning goal can be expressed by a constraint that the robot's position ends within the boundary of the tree's location. The planning goal for a more complicated trajectory that satisfies the instruction "fly to the tree, then fly to the lake" can be specified by a sequence of two constraints where the first constrains the robot to be at the tree and the second constrains the robot to be at the lake. With appropriate language, a navigation trajectory with many segments could be generated in this manner. A

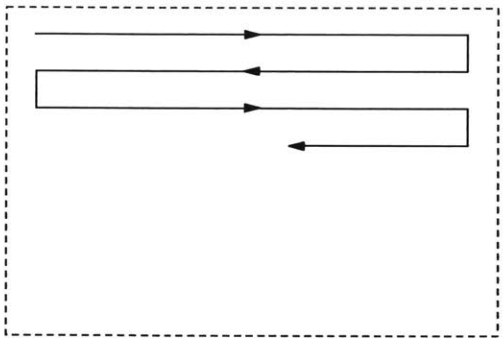


Figure 1-2: *A parallel sweep pattern* commonly used in aerial search.

search trajectory such as a standard parallel sweep pattern (Fig 1-2) could also be decomposed into a similar series of segments in terms of constraints at intermediate waypoints. However, this representation fails to capture the internal structure of the trajectory (i.e., regularly-spaced paths) and places an artificial importance on intermediate waypoints. In addition, the language used to specify the desired trajectories differs in scope; important intermediate waypoints are commonly specified in navigation but are generally not specified for search trajectories. A search commander may say "Search the field" but will not specify the component segments (e.g., "Fly along the edge of the field. Then turn right and fly 10 meters. Then turn right..."). Due to these differences in the input language and output trajectories, a new representation of the planning problem must be developed to use the probabilistic grounding graph approach in the outdoor search context.

Instead of grounding natural language to planning constraints, the alternative approach presented in this work involves grounding to a belief, or a probability distribution over the missing person's location. Works that seek to plan trajectories for aerial vehicles in outdoor search often assume the availability of a such a probability distribution. With access to a distribution over the unknown location of the missing hiker, a robot can use planning algorithms to choose a search trajectory that maximizes an appropriate objective function, such as the probability of finding the lost hiker within a given period of time. Since the beliefs involved in search and rescue are frequently irregularly shaped or multimodal (a person could choose to follow local

terrain or take one of multiple paths) the chosen belief representation must be capable of handling complex non-Gaussian distributions. To address this issue, I propose a set of *basis functions* to which language can be grounded. Each basis function represents a contributions to the belief, and the set of basis functions can be linearly combined to approximate irregular and multimodal beliefs.

In this work, I demonstrate that natural language commands can be grounded to beliefs by performing maximum likelihood inference on a distributed correspondence graph (DCG) [15]. The set of possible beliefs is infinite; I restrict the distributions to a large but finite subset generated by composing basis distributions defined relative to landmarks extracted from a map. To test the model, I generate pairs of natural language and corresponding beliefs using the Amazon Mechanical Turk crowd-sourcing platform. I quantitatively evaluate the effectiveness of the basis function approximation of the belief by comparing the probability of successfully finding the missing person for trajectories generated using both the belief inferred from the language instruction and a belief generated in a user study. Even with coarse approximations from simple language instructions, trajectories that are qualitatively reasonable and have a relatively high probability of successfully finding the missing person can be planned. The main description of this work can be found in Chapter 4.

1.2 Active Learning with Grounding Graphs

In the second part of this thesis, I use active learning to address the difficulty of acquiring sufficient training data to apply probabilistic grounding graph approaches in new environments. In practice, this training data may be time-intensive or expensive to obtain. For the probabilistic grounding graph method used here, the training data consists of pairs of language and groundings, where the groundings are represented in a symbolic form known to expert labelers but not to the non-expert labelers who are providing language for various scenarios. Non-expert labelers on crowd-sourcing platforms like Amazon Mechanical Turk can relatively inexpensively provide batches of language and scenario pairs when neither requires special knowledge of any un-

derlying representation. To be used as training data, however, the language needs to be annotated with the groundings in the underlying representation, a step that is tedious and requires expert intervention.

In general learning contexts where data is difficult to obtain, pool-based active learning strategies can be used to prioritize the labeling of examples in a pool according to their projected usefulness, with a goal of capturing the relevant information from the training data with a smaller but more informative labeled subset of all the possible data. In a language-based navigation scenario, some instructions may contain information that could be learned from other examples. For example, if the dataset in a language-based navigation scenario contains multiple instructions of the form "go to [a location]", additional examples of the same form may not provide new information, while a fundamentally different example such as "drive behind [a location]" would be expected to provide more information if labeled. In this work, I apply an entropy-based active learning approach based on uncertainty metrics in [42] and observe that the same performance on the test set can be achieved with fewer examples when using an active learning approach. The main description of this work can be found in Chapter 5.

1.3 Organization

This thesis presents two extensions to existing work in grounding natural language commands with probabilistic grounding graphs. First, I apply the probabilistic grounding graph approach in a new domain, that of outdoor search and rescue. The desired robot behavior for outdoor search and rescue is fundamentally different from the desired behavior in previous demonstrations, requiring a new set of groundings to be developed. Second, I address the difficulty of acquiring sufficient amounts of training data for training models in new domains by incorporating active learning into the probabilistic grounding graph approach. Appropriate data can be challenging to acquire in sufficient quantity due to the intensive nature of labeling data for grounding, and active learning allows the models to be trained with a smaller set

of expert-labeled data. As a third contribution, I also describe considerations for implementation, and provide a python implementation.

A survey of related works is presented in Chapter 2. The background on the probabilistic grounding graph method for grounding natural language is summarized in Section 3.1 and implementation considerations are presented in Section 3.2. The application of a probabilistic grounding graph method to inferring continuous belief spaces is detailed in Chapter 4. The use of active learning to reduce the quantity of labeled training data required to learn the model is detailed in Chapter 5. Concluding remarks and suggestions for future work are contained in Chapter 6.

Chapter 2

Related Works

This thesis presents a method for grounding natural language to beliefs for wilderness search and rescue and provides a strategy for reducing the annotation required for to train probabilistic grounding graph models. The thesis unites ideas from three main areas: natural language understanding for the direction of robots; trajectory planning for micro aerial vehicles in wilderness search and rescue; and active learning for structured prediction. An overview of related work in natural language grounding is given in Section 2.1. Section 2.2 presents related work in wilderness search and rescue, and Section 2.3 presents related work in active learning.

2.1 Grounding Natural Language Instructions

Significant progress has been made in enabling robots to infer intended actions from a language command in domains such as mobile robot navigation and manipulation. The SHRDLU system developed by Winograd [49] is considered an early effort at using natural language to direct a robot. Using a collection of programs that would parse language and reason about provided information, the system was capable of manipulating blocks on a simulated tabletop in response to typed commands in a limited environment. Harnad [12] first used the term *symbol grounding* to describe the problem of mapping words to meaning in an agent's internal representation.

MacMahon et al. [27] accepted unconstrained language and use the parse tree

structure of language to organize components for further translation to meaning. Trivedi et al. [45] proposed a constrained English command language with keyword pairs that map to an internal command structure. While they suggested indoor search and rescue as a possible use case, the constrained syntax required the humans to have prior training to communicate with the robot.

Shimizu and Haas [38] treated the indoor route-following problem as a sequential labeling task where instructions are split into chunks that correpsond to actions. They learned to map language to labels for a limited action frame representation with four fields that specify actions with respect to doors and hallways in an abstracted world. Kollar et al. [20] parsed language instructions into spatial description clauses, then used a probabilistic model to propose a sequence of waypoints through the environment. The subject, verb, spatial relation, and landmark fields of the spatial description clauses had a greater representational capability than the action frames of [38], allowing instructions to be interpreted relative to landmarks other than doors in a hallway. Their probabilistic model used the co-occurrence statistics of labeled images to ground to perceptual inputs and uses metric spatial features such as those in [40] to find paths most likely to match the spatial description clause. Huang et al. [16] used language instructions to direct a micro air vehicle. Their work built on previous work with spatial description clauses [20] by considering navigation in three-dimensional space, rather than two-dimensional space, and is demonstrated with a quadrotor using onboard sensors to fly through known space. Vogel and Jurafsky [47] used a reinforcement learning approach to learn the meanings of spatial terms in the context of route following in a two-dimensional drawn map with landmarks. These works assumed a flat language structure that does not extend to more complex instructions.

Matuszek et al. [28] and Chen and Mooney [4] assumed no prior linguistic knowledge and learned semantic parsers to follow route instructions in hallway environments from example pairs of instructions and corresponding paths. Matuszek et al. parsed to a formal language path description, and demonstrated their system in an environment with only automatic annotation. They did not make use of landmark

descriptions while navigating, while Chen and Mooney parsed to a formal navigation plan and focus on the use of landmarks such as artwork in their navigation task.

2.1.1 Probabilistic Grounding Graphs

Probabilistic grounding graphs are another approach to the symbol-grounding problem. Tellex et al. [41] presented the Generalized Grounding Graph (G^3) for inferring action trajectories from language by modeling a factor graph on the syntactic structure of the input instruction. A key limitation of the G^3 approach is scalability to real-life environments as learning a continuous trajectory representation becomes intractable. The Distributed Correspondence Graph (DCG) approach by Howard et al. [15] addressed this issue by inferring a set of planning constraints from language that allows a conventional planner to generate an action sequence, rather than inferring trajectories directly. An extension of DCG that increases the efficiency of search by first inferring a subspace of the most probable groundings to be considered was used in [14] and [13], and evaluated in [5]. Paul et al. [31] introduced a set of groundings for abstract concepts, allowing the model to reason over terms including groupings of objects, cardinality, and additional spatial references. Tucker et al. [46] addressed the problem of representing unknown concepts conveyed through language or objects detections a robot is unable to classify. Paul et al. [32] presented a temporally extended model that can propagate declarative and factual knowledge over time. Since the work in this thesis is also a probabilistic grounding graph approach, the core technical approach of this family of work is described in Chapter 3.

2.1.2 Other Recent Approaches

MacGlashan et al. [26] presented a model for learning latent reward functions within an Object-Oriented MDP representation that permits generalization to novel scenarios. Several recent works including Gopalan et al. [9], Shridhar and Hsu [39] and Gordon et al. [10] applied neural network approaches to the grounding problem.

Other authors worked more directly with spatial cost functions using simpler lan-

guage models. Boularias et al. [2] employed inverse optimal control to learn spatial cost functions for operational modes of a mobile robot given a collection of expert example trajectories. Sample et al. [34] used a Bayesian data fusion approach to enable human collaboration in a search for targets. Their work is similar to this work in that both generate probability density functions for target search in response to language input. However, since their focus is on the fusion of data from language and other sensors such as cameras, their language model is limited: language updates are restricted to a three-field fixed structure containing an object, a preposition, and a reference location. In contrast, this work does not use external sensor inputs but handles compositional and hierarchical elements of language.

2.2 Wilderness Search and Rescue

The goal of wilderness search and rescue is to find a missing person, or "target", as quickly as possible. In most cases, the search is driven by an effort to maximize the probability of success subject to resource availability. The probability of success in a region is usually modeled as the product of the probability that the target is in a region and the probability of detection, which is the conditional probability that the target will be found given that it is in the region being searched. The point at which the target was last seen, the target's expected trajectory, and visible signs of the target (such as clothing or other man-made items) concentrate probability in nearby regions, while the passage of time dilutes probability across an increasingly large region as the target may wander. The probability of detection is largely determined by the effective sweep width of a vehicle's sensor, which in turn depends on the altitude, speed, sensor efficiency over a given terrain, weather, and the visibility characteristics of the signs, etc. There is an inherent trade-off between coverage and detection, and increased time reduces the target's chances of survival, so search strategies must methodically concentrate on areas of higher probability and expand to regions of lower probability as efficiently as possible.

2.2.1 Aerial Vehicles in Wilderness Search and Rescue

The use of aerial vehicles extends the view of ground searches. Human observers can scan the landscape from piloted aircraft while unmanned vehicles can transmit image feeds for review. The trajectory of an aerial vehicle in piloted search is generally one of a set of template search patterns, such as an expanding square or parallel sweep pattern [50]. The appropriate search pattern is selected based on the size of the area to be searched, the confidence in the missing person's probable location or intended route, local geography, and other details of the incident. A number of works including [8, 29, 48] explored how micro aerial vehicles, rather than manned aerial vehicles, can be effectively incorporated into wilderness search and rescue teams.

2.2.2 Intelligent Paths for Searching with Micro Aerial Vehicles

The standard search trajectories assume very simple probability density functions; a parallel sweep pattern is efficient over a uniform distribution while an expanding spiral pattern is efficient over a unimodal symmetric distribution [8]. While a unimodal symmetric distribution can approximate a random walk from the point last seen and a uniform distribution may be reasonable when information is scant, in reality the probability distributions are more complex. Knowledge of the behavioral tendencies of lost persons can allow field experts to more accurately predict the probability density functions, including consideration of terrain effects and environmental factors [19]. Work has also been done to use data-driven Bayesian modeling [23] to help generate these probability distributions automatically from the given data. Trajectories planned on probability distributions that incorporate more data will perform better in expectation with respect to common metrics than trajectories that assume a simpler distribution. While search commanders may not be performing formal mathematical optimizations in the field, two common metrics considered when adding autonomy to the process are the mean time of search and the cumulative probability of success.

Bourgault et al. [3] presented a Bayesian approach to generating outdoor search

and rescue trajectories, in which a probability density function is predicted based on expected target behavior and subsequently updated with the probability of detection during observation. They optimized with respect to two different objective functions, mean time to detection and cumulative detection probability. Solving the optimization exactly with different finite time horizons, they found that a greedy one-step look-ahead method actually performed reasonably at low computational cost. Lin and Goodrich [22, 24] proposed and evaluated a number of approaches for fast approximate trajectory planning for outdoor search. In this work, we use one of the algorithms from [22] for planning trajectories.

2.2.3 Interfacing with Search Vehicles

Micro aerial vehicles can be piloted in a number of ways. Expert pilots can manually fly the vehicles. Lin and Goodrich developed and field tested a graphical user interface that allows pilots to specify waypoints and automatically generate basic search trajectories [25]. Flight planning tools that accept the necessary parameters for standard search trajectories through a graphical user interface and return the specified trajectory are readily available. If a voice interface were all that were desired, a speech-to-text system could be used to sequentially extract the parameters in an automated dialogue to plan a standard search trajectory. However, this interaction still requires an expert to provide the parameters, and aside from verbal interaction, does not provide the benefits associated with natural language interfaces. Supporting natural language would allow more natural commands such as "Search the field north of the lake" as opposed to specifying an initial coordinate, the initial heading, the spacing, and end condition in concrete units, enabling non-expert volunteers to control the vehicles and participate in the planning.

2.3 Active Learning

Dataset annotation can be a time-consuming endeavor in learning tasks such as speech recognition, information extraction, and classification. Oftentimes datasets contain

redundant information that take time to label but add little new information. The goal of active learning is to reduce the quantity of labeled data needed to achieve a high accuracy for a learned model by actively requesting labels for examples that are expected to be most informative. Many different heuristics have been used to estimate the value of a new label. The effectiveness of active learning methods is generally evaluated by comparing learning curves that show performance on a test set when models are trained on data selected by an active learning method versus trained on data that has been selected randomly or by another method.

2.3.1 Classical Active Learning

In a survey on active learning, Settles [36] identified three main active learning scenarios: pool-based sampling, stream-based selective sampling, and membership query synthesis. In pool-based active learning, the learner has access to a large pool of unlabeled data acquired at relatively low cost and a small initial set of labeled data. The learner can request labels from an oracle for additional examples in the pool. In stream-based selective sampling, unlabeled examples are sampled one at a time and the learner must decide whether to request a label or discard the example. In membership query synthesis, the learner may synthesize in the input space a new example for which it requests a label. This work uses pool-based active learning, where the pool consists of language and scenario pairs that have been collected through crowdsourcing. The learner can additionally ask the oracle to label new language examples from the pool with the grounding in an internal representation.

Many heuristics have been proposed to select the next examples for labeling, a number of which are mentioned here. Seung et al. [37] proposed Query by Committee, in which a committee of models that are consistent with existing data predict labels for future examples. The example for which there is the most disagreement is labeled. Lewis and Gale [21] proposed uncertainty sampling in a binary classification task, where the example for which the example with probability closest to 0.5 is labeled. In a multi-class setting Schein and Ungar [35] used both maximum Shannon entropy and minimum margin (difference between class probabilities) as measures of uncertainty.

Cohn et al. [6] selected data to minimize the average expected variance of the model after the new label is incorporated. Additional heuristics are described in literature surveys [36, 1], and a comparison of methods [35].

2.3.2 Active Learning for Natural Language, Grounding, and Structured Prediction

Many classical active learning works address classification tasks, where annotations are simply a category label. Many natural language learning tasks, however, including the grounding task formulated in Chapter 3, require more detailed annotation of complex structures. Thompson et al. [44] applied active learning to the tasks of natural language parsing and information extraction, where the annotations take on the form of sentence-and-representation pairs and filled templates. Since annotating a full new example provides information on multiple decisions within that example, they combine the value of labeling all decisions when determining the benefit of labeling a new example. In the context of activity recognition in images, Khodabandeh et al. [18] proposed a method of active learning where multiple labels in a single image are connected in a graphical model structure. Their active learning approach labeled the part of the image that causes the greatest expected average entropy reduction, making use of belief propagation to obtain necessary probabilities.

In other works with active learning and grounding, Pillai et al. [33] used active learning to train a joint model of language and perception for grounding language to objects in the environment. Active learning reduces the number of labeled examples needed to train the model as the learner can ask for full labels of items it does not know or ask for verification about labels for which it has low certainty. Thomason et al. [43] also used active learning in language grounding for object identification. They explored the benefit of requesting labels that are not immediately relevant for benefits in long-term performance. The language models in these works, however, are reduced to a bag of words model and descriptions of object properties, respectively, instead of a full natural language approach.

An entropy-based approach similar to [18] is appealing for this work due to the statistical and graphical natures of the probabilistic grounding graphs. However, due to the correspondence variable formulation of the probabilistic grounding graphs [41, 15] on which this work is based, the necessary conditional probability is not readily accessible. Instead, this work follows Tellex et al. [42], who formulated questions to clarify the least certain groundings in probabilistic graphical models. Tellex et al. used the readily available correspondence probability as a measure of certainty, and also approximate the entropy of the grounding's conditional probability using Bayes' rule. Active learning heuristics based on the correspondence variable certainty are sufficient for improvements in the case tested here.

Chapter 3

Probabilistic Grounding Graphs: Formulation and Implementation

Probabilistic grounding graphs are one main approach to the natural language grounding problem. This work builds on the probabilistic grounding graph methods [41, 15], so the approach is described in detail in this chapter.

Section 3.1 gives a conceptual overview and detailed mathematical formulation of the approach. Section 3.2 describes how the approach can be implemented.

3.1 Probabilistic Grounding Graph Formulation

3.1.1 Conceptual Overview of the Grounding Task

A grounding is an entity defined with respect to the physical world that satisfies the meaning of some input language expression. Consider a toy world $\mathcal{M}$ with just two objects, a lake and a tree. In this toy world, there are also only two relations that we can comprehend, *west of* and *near*. These four entities form a set of possible groundings $\Gamma = \{\gamma_1, \gamma_2, \gamma_3, \gamma_4\}$, as in Figure 3-1.

A subset of these groundings can be used to express an idea. For example, the subset

$$\{\gamma_1 = lake, \gamma_4 = near(\cdot)\} \tag{3.1}$$

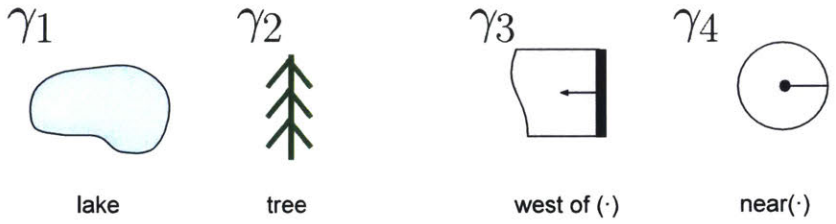


Figure 3-1: *Possible groundings in a toy world.* The symbols $\Gamma = \{\gamma_1, \gamma_2, \gamma_3, \gamma_4\}$ represent possible groundings in a toy world. Groundings γ_1 and γ_2 represent objects in the world model while γ_3 and γ_4 represent possible relations.

could be used to represent the meaning *near the lake*, as long as it is also understood somehow that γ_1 should be the argument of γ_4.

When grounding the text Λ = "near the lake" in the world model $\mathcal{M}$ that has possible groundings Γ, the groundings given in Equation 3.1 are ideal candidates. The grammatical structure of the text also provides additional information as to how the groundings should be assembled. The text "the lake" serves as the object of the preposition "near" to complete the prepositional phrase in English grammar. In the same way, the object $\gamma_1 = lake$ should be the object of the grounding $\gamma_4 = near$ to complete the concept *near the lake*. As the world models get larger and sentences get more complex, making use of the parse structure's additional information becomes even more important.

The goal in the grounding task is to determine which grounding symbols express the meaning of the language. Probabilistic grounding models attempt to learn the mapping from language to groundings from labeled examples, and use that model to predict groundings for new language. Probabilistic grounding graphs in particular have a structure that mimics the grammatical structure of the language to make the grounding process efficient.

3.1.2 Mathematical Formulation of Probabilistic Grounding Graphs

This section describes in detail how the probabilistic grounding graph methods on which this work is based formulate and solve the grounding task overviewed above. The Generalized Grounding Graph (G^3) [41] and the Distributed Correspondence Graph (DCG) [15] both generate factor graphs according to the structure of language, where each factor involves language elements λ, groundings γ, and correspondences ϕ. While these two works share many core ideas, the terminology and symbols have subtle differences that are explained below.

Core Formulation

The symbol grounding problem can be formulated as identifying the set of active groundings Γ^*_{active}, out of all the possible groundings Γ, that most likely correspond to the language Λ in the context of a world model $\mathcal{M}$:

$$\Gamma^*_{\text{active}} = \arg\max_{\Gamma_{\text{active}}} p(\Gamma_{\text{active}}|\Lambda, \mathcal{M}) \tag{3.2}$$

In the probabilistic grounding graph approach, the parse structure of the natural language is used to create a factor graph describing the likelihoods of different groundings. Each factor connects a known language constituent of the text ($\lambda \in \Lambda$), a grounding from the grounding set ($\gamma \in \Gamma$), and a correspondence variable $\phi \in \Phi$ representing whether the grounding matches the language. Variables of any child groundings (groundings of direct descendents of the language constituent in the parse tree) are also included in the factor. The set of correspondence variables Φ is introduced to allow both corresponding and non-corresponding pairs of language and groundings to be presented as examples for training. In this formulation, correspondence variable ϕ_i should be *True* when the grounding γ_i correctly expresses the meaning of the language λ_i, and should be *False* otherwise. Rather than denoting the groundings that are actively participating in the expression of meaning as Γ_{active}, the are indicated by a $True$ correspondence variable.

DCG Factorization

The DCG model treats the full set of groundings Γ as known, and seeks to find the maximum likelihood setting of Φ to indicate which groundings Γ are included in the representation of meaning. The new optimization for DCG including the correspondence variables is

$$\Phi^* = \arg\max_{\Phi} p(\Phi|\Gamma, \Lambda, \mathcal{M}). \tag{3.3}$$

In this formulation, identifying the maximum likelihood Φ is the same as identifying the most probable groundings.

Performing joint inference over all the correspondence variables Φ given all the language input Λ is intractible. Two assumptions are made in DCG to factor this large problem into multiple smaller problems. The first assumption is shared by G^3, which is that the structure of the parse tree captures the important relationships between the different parts of the language input, and that groundings of language constituents that are not directly connected are conditionally independent given the groundings between them. Following this assumption, the conditional probability can be factored using the chain rule and simplified using conditional independence into one term for each language constituent λ_i. This factorization can be written as

$$p(\Phi|\Gamma, \Lambda, \mathcal{M}) = \prod_{i=1}^{|\Lambda|} p(\phi_i|\gamma_i, \lambda_i, \Gamma_{c_i}, \mathcal{M}), \tag{3.4}$$

where Γ_{c_i} are the groundings of the child phrases of the language phrase λ_i.

The second assumption is only made in DCG, which is that each candidate grounding for a language phrase can be considered independently for inclusion. In the toy world example from Section 3.1.1, the inclusion of $\gamma_1 = lake$ into the grounding set would not affect the probability of $\gamma_2 = tree$ being included in the grounding set.

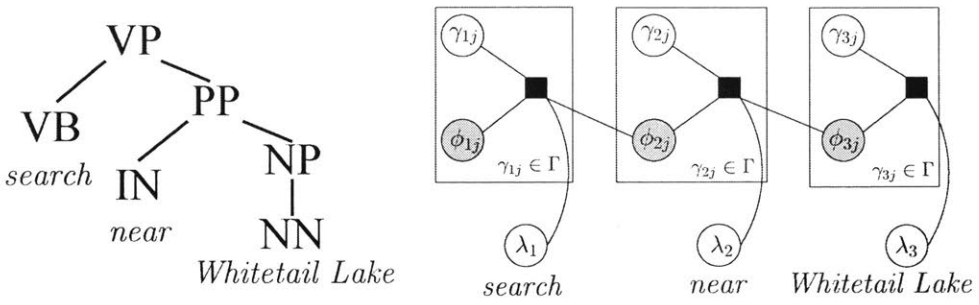


Figure 3-2: *Parse tree and corresponding factor graph.* The parse tree (left) and corresponding factor graph (right) are shown instantiated for the command "search near Whitetail Lake". Each factor connects a known grounding γ, a known word λ, an unknown correspondence ϕ between them, and the correspondence variables associated with children in the parse structure.

With this assumption, the conditional probability further factors into

$$p(\Phi|\Gamma, \Lambda, \mathcal{M}) = \prod_{i=1}^{|\Lambda|} \prod_{j=1}^{|\Gamma|} p(\phi_{ij}|\gamma_{ij}, \lambda_i, \Gamma_{c_{ij}}, \mathcal{M}) \tag{3.5}$$

where i indexes into the language phrases and j indexes into the possible groundings. Figure 3-2 shows the parse tree of a simple sentence and the DCG-style factor graph constructed according to its structure, with one factor per combination of phrase and grounding.

The Log-Linear Model

Each factor in Equation 3.5 is expressed using a log-linear model with binary features predicting association:

$$p(\phi_{ij}|\gamma_{ij}, \lambda_i, \Gamma_{c_{ij}}, \mathcal{M}) \propto \exp(\boldsymbol{\mu} \cdot \boldsymbol{f}(\phi_{ij}, \gamma_{ij}, \Gamma_{c_{ij}}, \lambda_i, \mathcal{M})). \tag{3.6}$$

Here, each element in $\boldsymbol{\mu}$ is the weight of the corresponding binary feature in the feature vector $\boldsymbol{f}(\cdot)$. Features include the association of language constituents with expression of a grounding and the relationships with child groundings. For more details on the features used, see Section 3.2.5. The weights $\boldsymbol{\mu}$ in the DCG model are trained using the limited-memory Broyden-Fletcher-Goldfarb-Shanno optimization algorithm (L-

BFGS) such that the weights of the features maximize the probability of example (Λ, Φ) pairs.

Differences between G^3 and DCG

The G^3 and DCG models actually diverge in approach after the correspondence variable has been introduced. In contrast to the DCG model, G^3 model treats the correspondences as constants with value $\Phi = True$ and seeks to find maximum likelihood values for its grounding variables Γ. The optimization for G^3 including the correspondence variables is

$$\Gamma^* = \arg\max_{\Gamma} p(\Phi|\Gamma, \Lambda, \mathcal{M}). \tag{3.7}$$

While DCG instantiates up to $|\Lambda||\Gamma|$ factors where each unknown ϕ_{ij} is binary valued (or ternary valued in cases where groundings can be rejected), G^3 instantiates $|\Lambda|$ factors, where each unknown γ_i has a domain that includes combinations of the elements of Γ.

Due to its roots in G^3, some notation and terminology used when describing DCG (and later work) is somewhat misleading. The phrase "inferring groundings" is often used to describe DCG's grounding process. While this terminology accurately describes the process in G^3, in DCG the values that are being inferred are not the grounding variables Γ, but rather, the correspondence variables Φ. The "inferred groundings" are a subset of fixed groundings whose associated correspondence variables have been inferred to be *True*. Also, since the values of Γ in DCG are set by deterministic functions on the world model $\mathcal{M}$, the term "variable" is no longer quite appropriate. For the purpose of being consistent with prior works, I use the terms "inferred grounding" and "child grounding" regardless of whether the groundings are being inferred directly, or active groundings have been identified by inferring corresponding variables. To understand this work, it is generally sufficient to accept that the groundings are inferred, and whether the inference is done directly or indirectly is an implementation detail. To implement this work, a deeper understanding is needed,

and additional information is provided in Section 3.2.

3.2 Implementation

This section is intended to give the reader a practical understanding of how the mathematical formulation from Section 3.1 translates into functional code. I also discuss some limitations of different implementations.

The software used for the first part of this thesis was adapted from the Human to Structured Language (H2SL) software[1] accessed in April 2016. The software used for the second part of this thesis was written in Python for easier modification. This section describes core organization and functionality, primarily using the Python version for examples, and highlights a few of their differences. Note that different versions of H2SL may be structured differently or provide different functionality.

3.2.1 Process Overview

There are two main tasks the software must perform: training, and inference.

The model training process begins with reading training files, which specify a world model, the correct parse tree of a string of text, and the groundings associated with each part of the parse tree. In both implementations described here, the inputs are XML files that are human-readable by expert labelers. The input from each file is converted to a tree of Phrase objects, where a Phrase object has fields representing the variables in the factor graph. A binary feature vector is computed for each Phrase based on the assignment of variables in the Phrase and its child Phrase(s), and this vector is assigned a positive label (indicating a true correspondence) in the data set. Examples with negative labels are not given by the expert directly, but are instead extrapolated based on the idea that other possible correspondences besides the given one are false. The log-linear model is trained using L-BFGS with these labeled feature vectors as input.

[1] https://github.com/tmhoward/h2sl

When a new instruction is received for grounding, it is first run through a parser to obtain a parse tree. The structure of the parse tree is used to make a tree of Phrase objects. During inference, binary feature vectors are computed for different pair assignments of text and candidate groundings, starting with leaf phrases in the parse tree and working toward the root. The probabilities of the pairings represented by the feature vectors are assessed by the model, and at each step a fixed number of pairings with the highest probability are propagated for consideration as children of phrases higher in the parse tree. Once at the root, the highest probability grounding is returned.

The rest of this chapter describes parses (Section 3.2.2), factor graph representations (Section 3.2.3), groundings (Section 3.2.4), features (Section 3.2.5), inference (Section 3.2.6), and training (Section 3.2.7). Inference is described before training because both processes make use of a routine that is more easily described in the context of inference.

3.2.2 Grammars and the Parse Tree

The probabilistic graphical model approaches used in this thesis have, at their core, a factor graph that parallels the *parse tree* of the natural language instruction. A parse tree shows the relationships between the parts of the sentence, where valid relationships are defined by a *grammar*. A detailed introduction to grammars and parse trees can be found in Jurafsky and Martin[17].

The Grammar

A grammar is a list of production rules that properly constructed text must obey. Different types of grammars place different restrictions on the types of rules allowed, such as the number of elements that can result from a single production. An example of a simple grammar that may handle simple navigation tasks in a toy home world is shown in the appendix (Figure A-1).

The grammar specifies valid combinations of *phrases* from which the input string S

can be composed. For example, an imperative sentence used as a command to a robot may be composed of a verb phrase followed by a noun phrase. Each of these phrases can produce more phrases or can produce *pre-terminal* parts of speech, such as verbs, prepositions, or nouns. These pre-terminals have unary production rules that produce single words, which are *terminal.* The process of parsing ends when a path has been found through the grammar from the input string to the terminal words and results in a series of productions that could generate the text. Note that complex language strings may have more than one parse that is consistent with the grammar—this ambiguity contributes to the challenge of working with natural language.

The Parse Tree

The parse tree of the language input captures the relationships between different language elements. Since the structure of the factor graph used for inferring groundings follows the structure of the parse tree, generating the parse tree is the first step when new language is received. Parsing is not the focus of this work, so I used a simple parser that is readily available.

The Natural Language Toolkit (NLTK) library for Python has parsing capabilities. The appendix contains code (Figure A-2) that uses a chart parser to find all parses of an input text string that satisfy a specified grammar, and optionally display the parse tree(s). The resulting parse tree for the text "go to the box" is shown in Figure 3-3. The particular parser used in the code example is very simple. While it is simple to implement and easy to modify the grammar, there are a few failure modes that more sophisticated parsing models could avoid.

First, if the parser encounters a word it has not seen before, or a sentence structure that is not specified by the grammar, the parse will fail and inference cannot even begin. This failure mode is particularly unfortunate when a simple synonym look-up or best-guess option would do well. Using a more robust parsing module would make the system more robust overall.

Second, this simple parser treats all possible parse trees that satisfy the production rules specified by the grammar as equally good. While there is no problem for simple

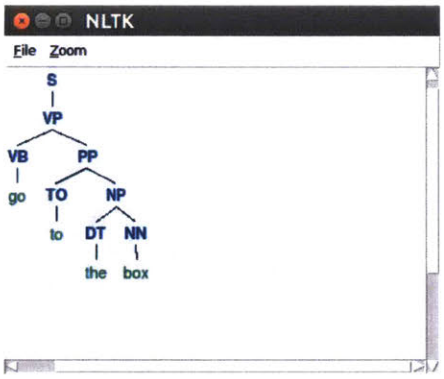


Figure 3-3: *Parse tree for the text "go to the box"* produced using the Natural Language Toolkit

sentences where the parse is unambiguous, longer instructions such as "search east of the woods north of the lake" will have multiple parses. Depending on the sensitivity of the model, different parses may result in different groundings. Selecting one parse at random may result in the wrong grounding, while processing all parses is time-consuming and may result in multiple outputs with no way to tell which is correct. Using a statistical parser, which can determine which of the valid parses is more likely based on training data, may have better performance. Despite these shortcomings, a simple parser can still give good results in many cases.

3.2.3 The Factor Graph Representation

Once a parse tree is acquired, the factor graph must be generated according to its structure. The probabilistic graphical model is a factor graph whose basic organizational unit is a Phrase. A Phrase contains a type (from the grammar), words (the text to be grounded), children (from the parse tree structure), and a grounding (to be inferred). Each Phrase corresponds to a factor in the factor graph. For bookkeeping purposes, a Phrase class may also contain a beam, which maintains candidate groundings during inference.

The Phrase could also have a field for correspondence variable value to denote whether the words and grounding valued assigned in the Phrase object are correctly paired. In my implementation, this field is not necessary as I accomplish that book-

keeping using vector labels during training and the beam field during inference. A sample Phrase class is presented in Figure A-3 in the appendix.

3.2.4 Groundings

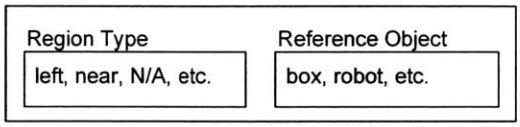
Groundings are objects, relations, constraints, and other ideas that have meaning in the physical world, and are represented by grounding symbols. Object groundings are straightforward: the word "the box" should be grounded to a symbol or group of pixels representing a box object that exists in the world. When grounding to a symbol, we assume that there is a way to map between these symbols and perceptual inputs from the real world. Groundings for relations or constraints require different symbols that do not necessarily correspond to physical objects in the world but can express their meaning. Different applications may also require different types of groundings to be expressible.

H2SL Groundings

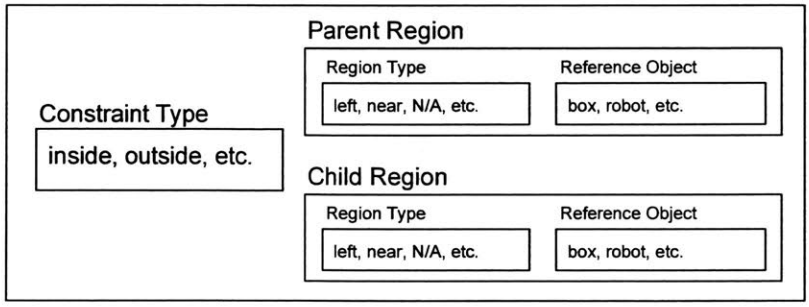
This section specifically describes the H2SL groundings available in the January 2016 version of the software, accessed in April 2016. In that verison, H2SL has two types of groundings: regions and constraints.

An H2SL region has two fields: a type and an object. The type could be *near*, *left*, etc., or a special type *N/A*. The object is an element of the world model such as a box or robot. To represent an object, a region with type *N/A* is used with the desired object in the object field. A pictorial representation of an H2SL region is shown in Figure 3-4a.

An H2SL constraint has three fields: a type, a parent region, and a child region. The type can be *inside*, *outside*, etc., and shows the relation between the two component regions. A planning constraint that the robot must be near the box is represented with a constraint of type *inside*, a parent region with type *N/A* and object *robot*, and a child region with type *near* and object *box*. A pictorial representation of an H2SL constraint is shown in Figure 3-4b. The symbolic representation of the grounding for



(a) H2SL Region



(b) H2SL Constraint

Figure 3-4: H2SL Groundings

an instruction that requires multiple steps is built from the leaves upward in Figure 3-5.

Basis Function Grounding Symbols The basis function groundings used in the first part of this thesis have the same form as the H2SL constraints. Rather than regions and constraints, the available grounding symbols in the search and rescue context are regions and basis functions. The constraint type field is replaced with a field for basis function strength. While only a single strength value was used in this work (the inference process only predicted whether a basis function should be activated or not), different values in the strength field could be used in future work to support different weights of basis function contribution based on certainty expressed in the language.

In theory, constraint and basis function groundings could co-exist, and with the right features, the model would learn that navigational words like "go" are more often related with constraints while coverage words like "search" are more often related with basis functions. However, a model with both types of groundings has not been implemented or tested.

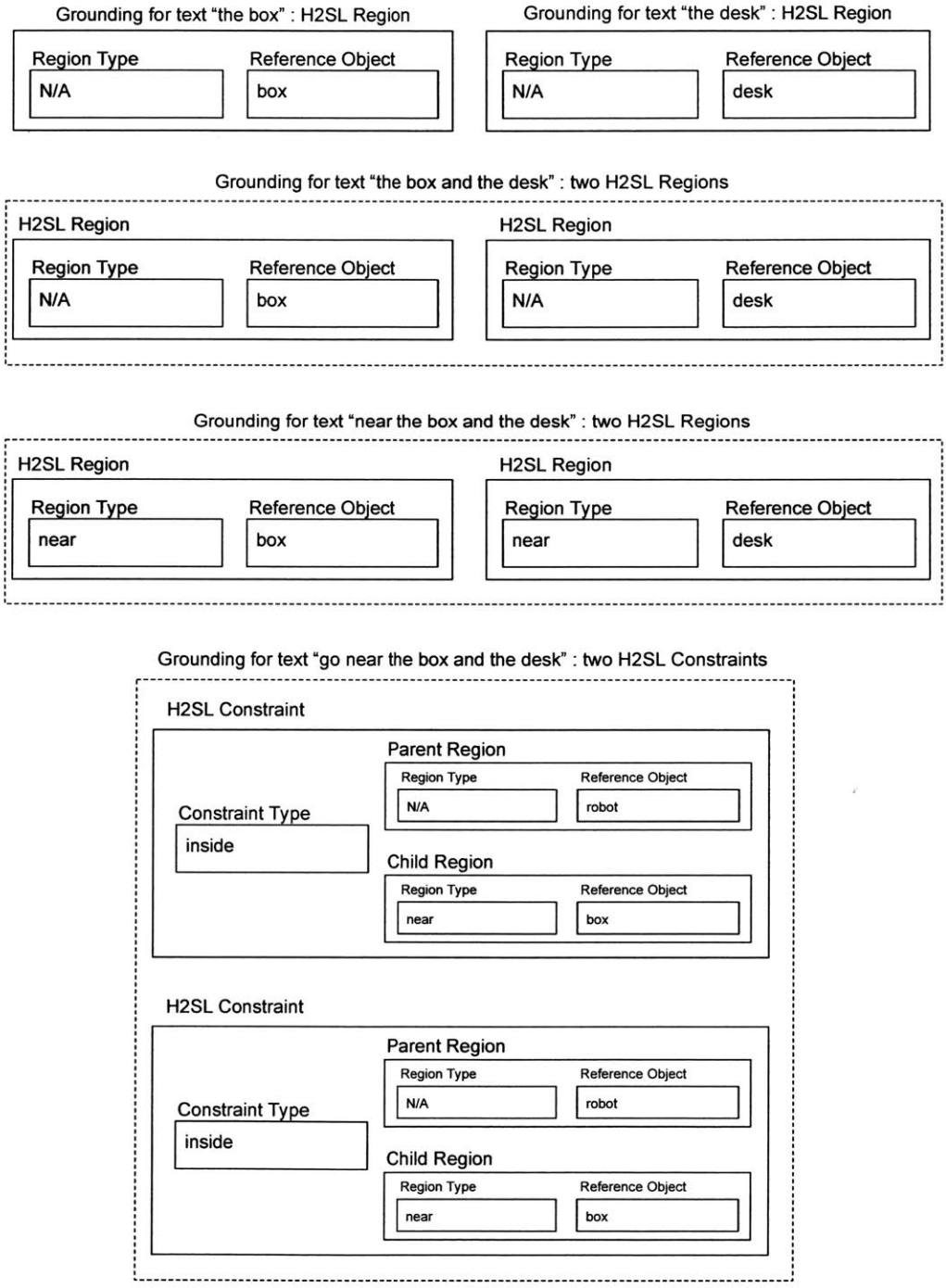


Figure 3-5: Grounding steps for "go near the box and the desk"

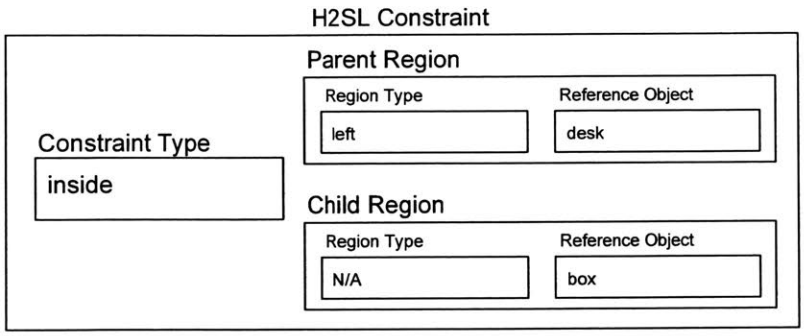


Figure 3-6: *Grounding for "Go to the box to the left of the desk".* The symbolic representation of this grounding does not explicitly include robot.

Limitations of Grounding Symbol Implementation The H2SL regions and constraints are simple and the number of combinations possible for a given constraint is countable. If the number of region types is $R \in \mathbb{N}$, the number of world objects is $\Omega \in \mathbb{N}$, and the number of constraint types is $C \in \mathbb{N}$, then the number of possible unique regions is $R\Omega$ and the number of possible unique constraints is $CR^2\Omega^2$. The values of R and C are generally small, so unless Ω is very large, enumerating the search space of single groundings is possible. (In this case, combinations of groundings and combinations with the child groundings are the source of computational load.)

One issue encountered using this representation in the outdoor search and rescue context is that only regions that are simply described by a single region type and reference object can be represented in full. There is no room in this symbolic form to include the multiple region types and multiple reference objects given by an instruction such as "search west of the woods north of the lake", or even "search the woods north of the lake". The same issue was encountered in earlier work that used H2SL, for navigation instructions like "Go to the box to the left of the desk". The solution there was to remove explicit reference to the robot in constraints, and use the constraint to represent the relation between two non-robot regions. The robot's goal of being in the resulting region was implied. An example of the top-level grounding for "Go to the box to the left of the desk" with this method is given in Figure 3-6.

Following earlier work, I also used this implied-robot representation in the basis function work. While the resulting groundings are intelligible, this solution is unsatis-

fying due to its inconsistency with simpler constraints and its inability to handle more complex descriptions should they arise. If H2SL used spatial metric information to resolve the nested descriptions to a single correct reference, a more complex grounding structure may not be necessary. When running H2SL in a purely symbolic mode, however, with metric information not incorporated until a post-processing step, all the information must be saved symbolically, and this approach is already at its limit. For the implementation used in the active learning work, I use a different grounding structure that allows nesting of groundings to make use of the additional information. Details on the new grounding structure can be found in Section A.3.1 in the appendix.

3.2.5 Log-Linear Model Features

Log-linear models are commonly used in natural language processing. In a log-linear model, the logarithm of the conditional probability of a label $y \in Y$ given some input $x \in X$ is proportional to a linear combination of feature values computed for that x and y. That is,

$$p(y|x;\boldsymbol{\mu}) \propto \exp(\boldsymbol{\mu} \cdot \boldsymbol{f}(x,y)) \tag{3.8}$$

A good tutorial on using log-linear models in language applications can be found in Collins [7].

In the natural language grounding tasks, the labels are the correspondence variables $\phi_i \in \Phi$ and the inputs include the language $\lambda_i \in \Lambda$, candidate groundings $\gamma_i \in \Gamma$, and child groundings Γ_{c_i}. The binary feature vectors $\boldsymbol{f}(\phi_i, \gamma_i, \Gamma_{c_i}, \lambda_i, \mathcal{M})$ have been designed to highlight co-occurrences between properties of the language and the groundings. Two separate sets of base features are defined, one set that focuses on language properties and one set that focuses on grounding properties. Their Cartesian product, which pairs each language feature with each grounding feature, forms the set of feature vectors that are actually used to train and query the model.

Language and Grounding Base Features

The first set of base features is defined to highlight language elements, such as whether a specific word or part of speech is present in a Phrase. For example, a base feature `word_box` will have value 1 when the word "box" is in a Phrase, and will have value 0 if it is not. If the word "box" is indeed in the Phrase, we could also expect a feature for the noun part of speech, `pos_noun`, to take value 1.

The second set of base features is defined to highlight grounding elements, such as whether a certain object or relationship is involved in the grounding of a Phrase. If a box object is involved in the grounding of a Phrase, then a base feature `obj_box` should take value 1. If the grounding instead involves a desk object, the value of the feature `obj_box` would be 0.

The set of grounding base features also include features that compare the grounding of the child Phrase to the grounding of the current Phrase. For example, a feature `grounding_matches_child_grounding` will have value 1 if the grounding of the child Phrase is identical to the child of the current Phrase, and 0 otherwise. This feature can help learn how groundings are propagated up from leaf Phrases toward the root. For the lists of base language features and grounding features used in H2SL, see Section A.3.2.

Metric Spatial Features The language and grounding base features mentioned thus far do not make any use of the coordinates of the groundings. While other works explore mathematical representations of spatial relationships in language grounding (e.g., [40]), neither implementation used in this work includes metric spatial features. In this work, if the text specifies "the box to the left of the desk" and there are boxes on either side of the desk, the grounding features listed above are insufficient to interpret which box is intended based on their coordinates. The output of the inference algorithm with these features is a symbolic representation specifying a box object with property *left of* with respect to a reference desk object, as in Figure 3-6. This symbolic representation needs to be interpreted in metric space by a post-processing step.

In a cleaner implementation, metric spatial features should be included as grounding features. These features would consider the difference between the coordinates of the target object and reference object, for example. Then the model could learn to relate words like "left" or "west" to coordinate differences in the robot or world coordinate frame.

The Feature Product

As mentioned in the overview of the feature vectors, the language and grounding base features are not used alone, but are rather combined through a Cartesian product to form the full set of vectors. If there are $L \in \mathbb{N}$ language base features and $G \in \mathbb{N}$ grounding base features, there are LG features in the full vector. Combining the language base feature `word_box` with the grounding base feature `object_box` results in an H2SL feature `word_box_&_obj_box`.

While the feature vector is large, it is also quite sparse. For readability purposes, the feature vectors in pyh2sl can easily be switched between a binary vector and a human-readable text listing of the features that are activated. Two features in such vectors could be `word_box_&_obj_box`, which we expect to have a high weight in the model since the word "box" should correspond to a box object, and `word_box_&_obj_desk`, which we expect to have a low weight in the model because the word "box" should not correspond to a desk.

3.2.6 Inference

The inference step seeks to find the grounding with the highest conditional probability for the given instruction and the world model. The probability is assessed based on the feature vectors associated with the language input's Phrases, so language input is first parsed and represented by Phrases.

Beam Search

As mentioned in Section 3.2.4, there are a finite number of possible H2SL groundings that can be considered for any particular Phrase once the world model has been specified. The H2SL implementation enumerates possible region groundings by looping over region types and objects, and enumerates possible constraint groundings by looping over constraint types and regions. While proposing each of these groundings is feasible, the features depend not only on the current Phrase's grounding, but also on the grounding of its child Phrase(s). Considering all combinations of groundings of the current Phrase and its child Phrase(s) is cost prohibitive, so a beam search is used to limit the number of child groundings under consideration. With a beam width of size k, the top k groundings from a child Phrase are considered in turn as possibilities for the grounding of the child Phrase when attempting to ground the current Phrase.

Using the same notation as in Section 3.2.4 where the number of region types is $R \in \mathbb{N}$, the number of world objects is $\Omega \in \mathbb{N}$, and the number of constraint types is $C \in \mathbb{N}$, the the number of combinations considered is $k(R\Omega + CR^2\Omega^2)$. With a beam size of 1, beam search reduces to greedy search, where the most likely grounding is accepted at each step.

For each combination of current and child Phrase groundings generated in this manner, the associated feature vector is computed and given to the model for evaluation. Ignoring the constant of proportionality, which is the same for all combinations, the probabilities can be compared using the value of $\boldsymbol{\mu} \cdot \boldsymbol{f}(\lambda_i, \gamma_i, \Gamma_{c_i}, \mathcal{M}, \phi_i)$, which I refer to as the "score". The k candidates with the top scores are stored within the beam and are used as possible child Phrase groundings for the next Phrase to be grounded.

Due to the structure of the factor graph that allows $p(\Phi|\Gamma, \Lambda, \mathcal{M}; \boldsymbol{\mu})$ to be factored as $\prod_i p(\phi_i|\gamma_i, \Gamma_{c_i}, \lambda_i, \mathcal{M}; \boldsymbol{\mu})$, the probabilities of larger and larger subtrees can be acquired by multiplying the probabilities of factors in the subtrees. The scores, which are unnormalized log-probabilities, can simply be added when used for comparison.

A Note on Different Models

In H2SL as based on the Distributed Correspondence Graph (DCG) model, if the language corresponds to a set of multiple groundings, the membership of each element grounding is determined independently—all groundings that score over a threshold are included as part of the grounding. In the Generalized Grounding Graph (G^3) model, only the single highest probability grounding is considered to be the grounding. If the language should ground to a set with more than one element, these sets need to be proposed as candidate groundings for evaluation. My Python implementation follows the G^3 model.

The difference between these models is highlighted when there are multiple objects of the same type in the world model. If the world model contains two identical boxes and the text to be grounded is "box", DCG will return both boxes since each will individually meet the threshold to ground to "box", while G^3 will pick a single box, likely breaking the tie randomly as it tries each possible grounding in turn. In the same world model, if the text to be grounded is "boxes", DCG will again return both boxes, while G^3 will return both boxes only if sets of objects are considered as possible groundings in addition to individual objects.

3.2.7 Training

Recall that the form of the model is

$$p(\phi|\gamma, \lambda, \mathcal{M}; \boldsymbol{\mu}) \propto \exp(\boldsymbol{\mu} \cdot \boldsymbol{f}(\lambda, \gamma, \gamma^c, \mathcal{M}, \phi)). \tag{3.9}$$

The values for the weights $\boldsymbol{\mu}$ are computed during training to maximize the product of the probabilities of all the training examples (or the sum of the log probabilities) subject to L2 regularization on the weights.

Training data that shows correspondences between language and groundings is necessary to train the model. Annotators on crowd-sourcing platforms like Amazon Mechanical Turk can provide pairs of language and scenarios as described for the search context in Section 4.5. An expert labeler can further annotate the data using

the underlying representation needed for training. In addition to examples with correct correspondences, examples with incorrect correspondences need to be provided for the model to be trained. These additional examples can be extrapolated from the positive examples.

Expert-Labeled Positive-Label Training Data

Both implementations in this work accept training examples in an XML format. Excerpts from a sample training file can be found in Section A.4 in the appendix. Figure A-5 shows the instruction "go to the box" that is to be carried out in a world model containing a robot, a box, a desk, and a chair. In a minimal example, only the presence of an object needs to be denoted. Other properties of the objects could be added to the world model to allow features based on position, color, mass, etc.

The groundings shown schematically in Section 3.2.4 are also provided in the XML format in the expert-annotated training files. Figure A-6 in the appendix shows the grounding set of the top-level verb phrase for the instruction "go to the box". For this instruction, there is only one constraint at the top level, which follows the format described in Section 3.2.4, where a robot object is constrained to be inside a box object's immediate vicinity. More complicated instructions could have multiple constraints in the top-level grounding set. The annotation for the child phrases (i.e. the prepositional phrase "to the box" and noun phrase "the box") are not shown in Figure A-6.

The XML structure provides the intended parse structure, so a parser is not needed during training. Each linguistic phrase in the XML can be directly translated into a Phrase object, with the text and grounding provided, the child phrases identified by the structure, and the correspondence understood to be True. Feature vectors computed from these examples are assigned a positive label for training. Since an instruction contains multiple phrases, each training instruction provides multiple positive-label examples for training.

Extrapolating to Negative-Label Examples

For every correct correspondence of language and groundings, there are overwhelmingly more language and grounding pairs that do not correspond. To scope the negative-label examples to those that may be most useful during inference, a routine from the inference process (Section 3.2.6) is actually used to generate incorrect examples during training. During inference, a large number of possible groundings are proposed for consideration. During training, the same procedure is used to propose candidate groundings. Feature vectors computed from any grounding assignment besides the one provided as correct in the training example are assigned a negative label for training. The sheer quanitity of possible groundings still poses an issue, however. During inference, the candidate groundings would be ranked by likelihood according to the trained model and pruned to the the size of the beam. During training, there is not yet a trained model that can score and rank the groundings, but the full enumeration is still too large to propagate up through the parse tree. In my implementation, the beam is filled with a random selection of candidate groundings and the beam is made as large as is feasible to maintain the most examples. In this way, both positive- and negative-label vectors can be obtained from the same expert-labeled data.

Optimization

In Python, the scikit-learn LogisticRegression class[2] can perform L2 regularization for log-linear models using L-BFGS. Code provided in Figure A-7 in the appendix will return a trained model that can be queried with new vectors to obtain scores.

3.3 Summary

This chapter described the mathematical formulation of grounding natural language instructions using probabilistic grounding graph methods and presented considerations for its implementation. Chapter 4 describes an extension to this work that supports the use of this model for inferring beliefs from natural language. Chapter 5

[2]http://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

describes an extension to incorporate active learning with this method to reduce the quantity of annotated training data required.

Chapter 4

Grounding Natural Language to Beliefs for Search and Rescue

When a person goes missing in the outdoors, a search crew can use a micro aerial vehicle to assist in scanning the area. This chapter describes a way for the vehicle to be directed to produce reasonable search trajectories using natural language commands. The language is first used to approximate the belief of the missing person's location and a trajectory is subsequently planned on that belief.

This chapter is organized as follows: Section 4.1 outlines the process. Sections 4.2, 4.3, and 4.4 give additional detail on grounding natural language to beliefs, the basis functions used to represent beliefs, and how trajectories are planned given those beliefs. Section 4.5 describes the data used to evaluate and train the model, and Section 4.6 describes experimental results.

4.1 Technical Approach Overview

The goal in this search and rescue scenario is to find the location $\mathbf{x}$ of a missing person given a natural language command Λ in the context of a world model $\mathcal{M}$. This problem is addressed by first inferring a belief $p(\mathbf{x}|\Lambda, \mathcal{M})$. Then, conditioned on that belief, a contiguous sequence τ of robot positions $\mathbf{x}_r$ is found to maximize the probability of finding the target within a fixed time window T. When planning a

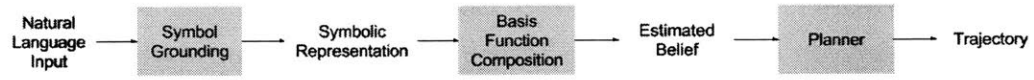


Figure 4-1: *Approach Overview.* The process from natural language command to search trajectory is composed of three steps. In the first step, the Distributed Correspondence Graph is used to infer the correct association between the input language and symbols representing basis functions of a spatial distribution in a process known as *symbol grounding.* These basis functions are defined such that together they specify a probability distribution of a missing person's location. Then a planner generates a trajectory over the belief.

search trajectory, a perfect detection model and a discrete world model are assumed, so that the probability the robot detects the target is

$$p(\text{detection}|\mathbf{x}, \mathbf{x}_r) = \begin{cases} 1 & \mathbf{x} = \mathbf{x_r} \\ 0 & \text{else} \end{cases}. \tag{4.1}$$

The meanings of the input language instructions Λ are inferred in the process of *grounding*, resulting in a symbolic representation in terms of correspondence variables Φ and grounding variables Γ that relates to continuous spatial beliefs (described in detail in the subsequent section).

Data collection for this task indicated that annotators typically describe beliefs and commands with respect to map landmarks. Consequently, a belief representation has been defined in terms of basis functions $b_k \in \mathcal{B}$ that are tied to landmarks. The probability of finding the missing person at location $\mathbf{x}$ given a map $\mathcal{M}$ and symbolic grounding variables Φ, Γ is then given by

$$p(\mathbf{x}|\mathcal{M}, \Phi, \Gamma) \propto \sum_k \phi_k b_k(\mathbf{x}) \tag{4.2}$$

where $\phi_k \in \{0, 1\}$ is an element of Φ and each k is related to a landmark.

The high level components of this system are shown in Figure 4-1. Additional detail on the components can be found in the Sections 4.2, 4.3, and 4.4 .

4.2 Grounding Natural Language Instructions

The first step toward acting on a given natural language command is to *ground* the text Λ of the command to a symbolic representation in terms of correspondence variables Φ and grounding variables Γ that identifies the belief denoted by the language. The text $\Lambda = \{\lambda_1, \ldots, \lambda_n\}$ is comprised of a set of language constituents λ_i. Grounding is the process of determining the meaning of the language constituents by associating them with objects or concepts in the real world. Possible objects or concepts are referred to as *groundings* $\gamma \in \Gamma$ in general. The specific grounding γ_i that corresponds to the language constituent λ_i is identified with a true correspondence $\phi_i = True$. The space of groundings depends on a world model $\mathcal{M}$, which consists of the type and metric extents of all the known landmarks in a region. For example, a landmark representing a lake would have type "lake" and an extent represented by the coordinates of a polygon within the map. In this work, the world model $\mathcal{M}$ and set of all possible groundings Γ are assumed to be fully known.

As an example of grounding natural language for search and rescue, consider the symbolic representation for the command, "Search near Whitetail Lake". The text parses into a noun (NN = "Whitetail Lake"), a prepositional phrase containing that noun (PP = "near" NN), and, at the highest level, a verb phrase (VP = "Search" PP). The grounding for "Whitetail Lake" is a specific lake object in the map, and the grounding for "near [Whitetail Lake]" is a region near that lake object grounding. Since "Whitetail Lake" is a child phrase of the prepositional phrase in the parse tree, and since the grounding of "near [Whitetail Lake]" depends on the grounding of "Whitetail Lake", the Whitetail Lake object is a *child grounding* of the prepositional phrase. At the root level, the verb phrase grounds to the basis functions that correspond to the child grounding, the region near Whitetail Lake. More complex examples can include composition (e.g. multiple regions) and deeper hierarchies (e.g. directions relative to other regions).

In this work, the H2SL software, which is a simplified open-source version of the Distributed Correspondence Graph (DCG) [15] (see Section 3) is used to ground the

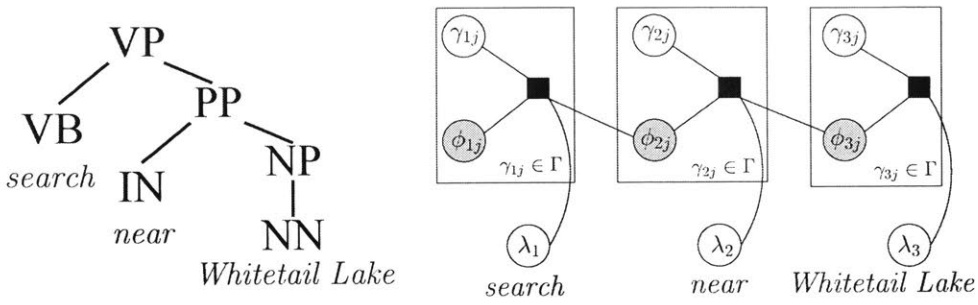


Figure 4-2: *Parse tree and corresponding factor graph.* The parse tree (left) and corresponding factor graph (right) are shown instantiated for the command "search near Whitetail Lake". Each factor connects a known grounding γ, a known word λ, an unknown correspondence ϕ between them, and the correspondence variables associated with children in the parse structure.

commands to beliefs. For the mathematical formulation of the approach, see Section 3.1. For a description of the implementation, see Section 3.2.

4.3 Basis function composition

In its initial presentation, DCG was used to ground natural language commands to planning constraints separating the world into admissible and inadmissible regions. The primary modification in this work is to replace the planning constraint groundings with basis function activation groundings. This section describes a basis function representation of the possible beliefs.

4.3.1 The Basis function

To create the basis functions, landmarks and regions of interest in a given map are segmented by hand; although it is not the focus of this research, this process could be automated using object detection techniques. The basis distributions are designed to reflect the shape of the underlying landmarks while still taking into account uncertainty near the landmark boundary. For example, to model the grounding for the region "near Whitetail Lake", Gaussian smoothing is applied to a binary image whose pixel values are 1 over the lake and 0 elsewhere. To address the range of language that may be encountered (e.g. "north of", "near", etc.), multiple bases are defined

for each landmark. To model the grounding for the region "west of Whitetail Lake", the bounding box of the polygon defining the extents of the lake is reflected across the western side of its bounding box, and the Gaussian smoothing is applied to the resulting binary image. The correspondence variables ϕ determine which bases should be activated, given the language. Once all contributions to the belief are identified, result of the smoothing is normalized into a valid probability distribution.

4.3.2 Composition

In the implementation used in this work, the grounding process is entirely symbolic—the output of the grounding process is a symbolic representation that must still be converted to actual probability values at coordinate locations. While increasing the feature space to include metric considerations may allow for more of the metric interpretation to be handled within H2SL, in this work, a simple mapping from the symbolic representations inferred to H2SL to the probability distributions used to plan trajectories has been hand-specified.

When the grounding is a single basis function, the probability distribution described above is used. When multiple bases are activated by a grounding, the basis contributions are summed. To model a region that must be disambiguated from other similar landmarks in the map by its relation to a parent region, the local basis function for each region of the appropriate type is included, weighted by how well the region satisfies the specified relation. For example, for the region "the archery range west of Whitetail Lake", a weight is computed for each archery range using geometric features such as the distance and relative direction to the centroid of Whitetail Lake averaged over the archery range's area. Archery 2, which is west of and near Whitetail Lake, will have a high weight given this grounding, while Archery 1, which is southwest and far from Whitetail Lake, will have a weight near zero.

Learning appropriate weights for the basis function contributions from the language and map characteristics is an area for future work.

4.4 Trajectory Generation

Once the maximum likelihood belief has been obtained, a planning problem to maximize the probability of successfully finding the target subject to a time constraint can be formulated and solved. Because solving this planning problem directly is computationally intractable, a number of authors [3, 11, 22] explore approximate algorithms for solving the optimal search problem, with additional work addressing different detection models and target behavior. This work uses an approximate planner developed by Lin and Goodrich [22]; this planner, based on local hill climbing, was found to have a shorter run-time and comparable search efficiency in a comparison of approximate algorithms.

In this work, a stationary target and perfect detection of targets within the robot's view are assumed. Under these assumptions, the region covered by a trajectory $\tau = \vec{\mathbf{x}}(t_i, ..., t_f)$ is

$$\mathcal{V} = \bigcup_{t=t_i}^{t_f} \text{vis}(\mathbf{x}(t)) \tag{4.3}$$

where $\text{vis}(\mathbf{x})$ is the region visible to the robot at location $\mathbf{x}$. The objective J_{POS} representing the overall probability of success with trajectory $\vec{\mathbf{x}}(t_i, ..., t_f)$ is

$$J_{\text{POS}}(\vec{\mathbf{x}}(t_i, ..., t_f)) = \int_{\mathcal{V}} p(\mathbf{x}) d\mathbf{x} \tag{4.4}$$

which integrates the probability density function over the full region viewed.

For the planning step, the inferred probability distribution is discretized and a finite probability is assigned to each grid cell on the map. The robot is assumed to be able to move to an adjacent cell in time Δt and perfectly detect whether the target is present. With these assumptions, the probability of success is simply the sum of the probability values in each of the cells through which the robot trajectory passes. This metric is equivalent to collecting a one-time reward whenever a new cell is visited, such that the probability of success can be written as the sum of a function p' over

the path,

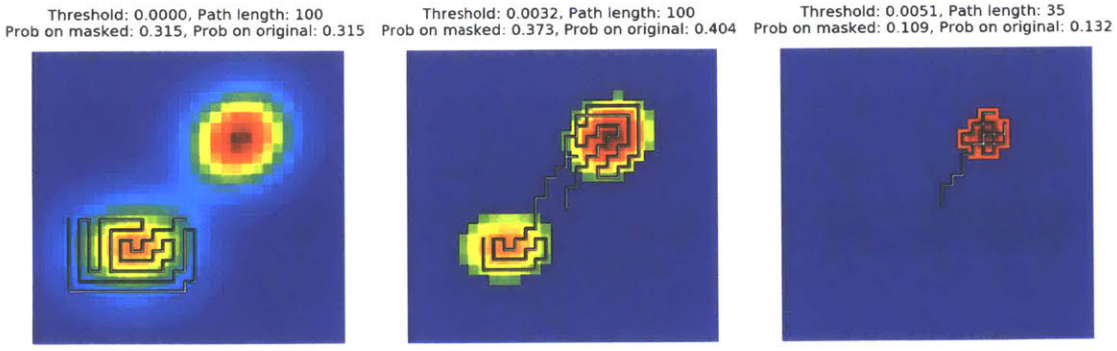
$$J_{\text{POS}}(\vec{\mathbf{x}}(t_i, ..., t_f)) = \sum_{t=t_i}^{t_f} p'(\mathbf{x}_t) \tag{4.5}$$

where

$$p'(\mathbf{x}_t) = \begin{cases} 0 & \text{if } \mathbf{x}_t \in \{\mathbf{x}_0, \mathbf{x}_{\Delta t}, ...\mathbf{x}_{t-\Delta t}\} \\ p(\mathbf{x}_t) & \text{otherwise.} \end{cases} \tag{4.6}$$

In a local hill climbing approach, the robot moves greedily to its most probable neighbor and receives a one-time reward of $p(\mathbf{x}_{\text{neighbor}})$. Under the assumptions of perfect detection and a stationary target, the probability of success is not improved by revisiting a grid cell, so the reward of a cell is set to 0 when the cell is first visited. If all the cells neighboring the robot have probability 0, the robot moves in the direction of the global maximum. If all cells in the map have probability 0, search terminates.

A simplistic greedy approach is unsuitable in many real search and rescue scenarios, however, such as when a bimodal distribution is to be searched within a time limit. Where a more intelligent approach would split time between the two modes to explore the most likely regions of each mode, a greedy planning approach may cause a robot to waste time fully exploring the low probability regions of one mode and fail to explore the second mode at all. The implementation used here following Lin et al. [22] addresses the issue by removing the lower values of the map in sequential runs of the greedy algorithm and forcing the robot to seek out farther but higher probability locations sooner. Local hill climbing is run on n different masks of the map, each with probabilities under a different threshold set to 0. The greedy trajectory from each masked map is then evaluated on the original map, and the trajectory with the highest probability of success on the original map is returned. Three illustrative threshold levels for planning on a bimodal distribution are shown in Figure 4-3. Each greedy search is fast, so running the search multiple times at different thresholds is not a significant computational burden. Twenty equally spaced threshold values are used for each plan in each experiment.



(a) No threshold (b) Moderate threshold (c) High threshold

Figure 4-3: *Planning example for maximizing probability of success on a bimodal distribution.* A greedy planner used in conjunction with a repeated thresholding procedure presented by Lin & Goodrich [22] allows the generation of reasonable trajectories at low computational cost in cases where a purely greedy planner will fail. Trajectories are planned with 20 threshold values and the resulting trajectory with the highest probability of success on the full distribution is selected.

4.5 Human-Annotated Beliefs

To evaluate the quality of the beliefs generated from the natural language command, pairs of beliefs based on different missing-person scenarios and language describing where to search based on those beliefs are required. This section describes the generation of this data set using three rounds of data collection using the crowd-sourcing platform Amazon Mechanical Turk (AMT).

First, annotators were given the blank camp map in Figure 4-4 and a series of prompts asking them to provide text describing some details of a missing person scenario. For example, statements of the missing person's intent, tendencies, and point last seen could explain why a person is likely to be found in a particular region. This step generated descriptions of 80 scenarios in which a camper has gone missing.

Next, annotators were provided with the text of a scenario and were asked to highlight where a rescuer should search in three levels of priority, providing rough level curves on the probability distribution of the missing person's location for each scenario (see Figure 4-5). The submissions from all 5 respondents per scenario were combined with a weighted sum (weights 3, 2, 1, for priority levels 1, 2, 3, respectively), and smoothed with a Gaussian filter to provide the final human-annotated probability

density functions (PDFs) for the scenarios, which are treated as ground truth. The rationale behind asking new annotators to highlight where to search rather than asking the original annotator to provide both the text and the highlighting is so that the highlighted regions match the information provided about the scenario rather than some unexpressed idea that the originator of the scenario may have held.

Finally, pairings of beliefs of missing person location and language describing where to search were generated by providing annotators with the consensus belief for a scenario and asking for instructions to an English-speaking robot on where to search. Since the complexity of sentences supported directly by the built-in features of the open-source version of the H2SL software is limited, annotators were requested to provide both a concise instruction that captures a large part of the probability mass (and would likely be groundable by H2SL), and a detailed instruction that fully captures the highlighted region (but would likely not be groundable by H2SL). Of the 240 concise commands collected, 103 unique commands (127 including duplicates) could be grounded with negligible modifications to open-source H2SL. Grounding more detailed commands will require an expansion of the grammar and the development of additional features for H2SL, and is an area for future work. Sample responses for the concise and detailed instructions include "Search near the soccer goals" and "Search the soccer field to the east, as well as the Alexander Trail and skeet shooting range to the south and west", respectively (see Figure 4-6). Additional corpus examples are shown in Figure 4-7.

4.5.1 Expert-Annotated Training Data

The human annotators who provide the language and belief pairs do not have access to the internal representation of the grounding problem. Preparing data for training the model requires an extra processing step by expert annotators to label the groundings at each phrase in the parse tree (see Section 3.2.7).

The training examples consist of two main types that were hand-selected by the expert annotator. One set of examples contains full commands with respect to generic symbolic objects to learn compositional and hierarchical structures in the language

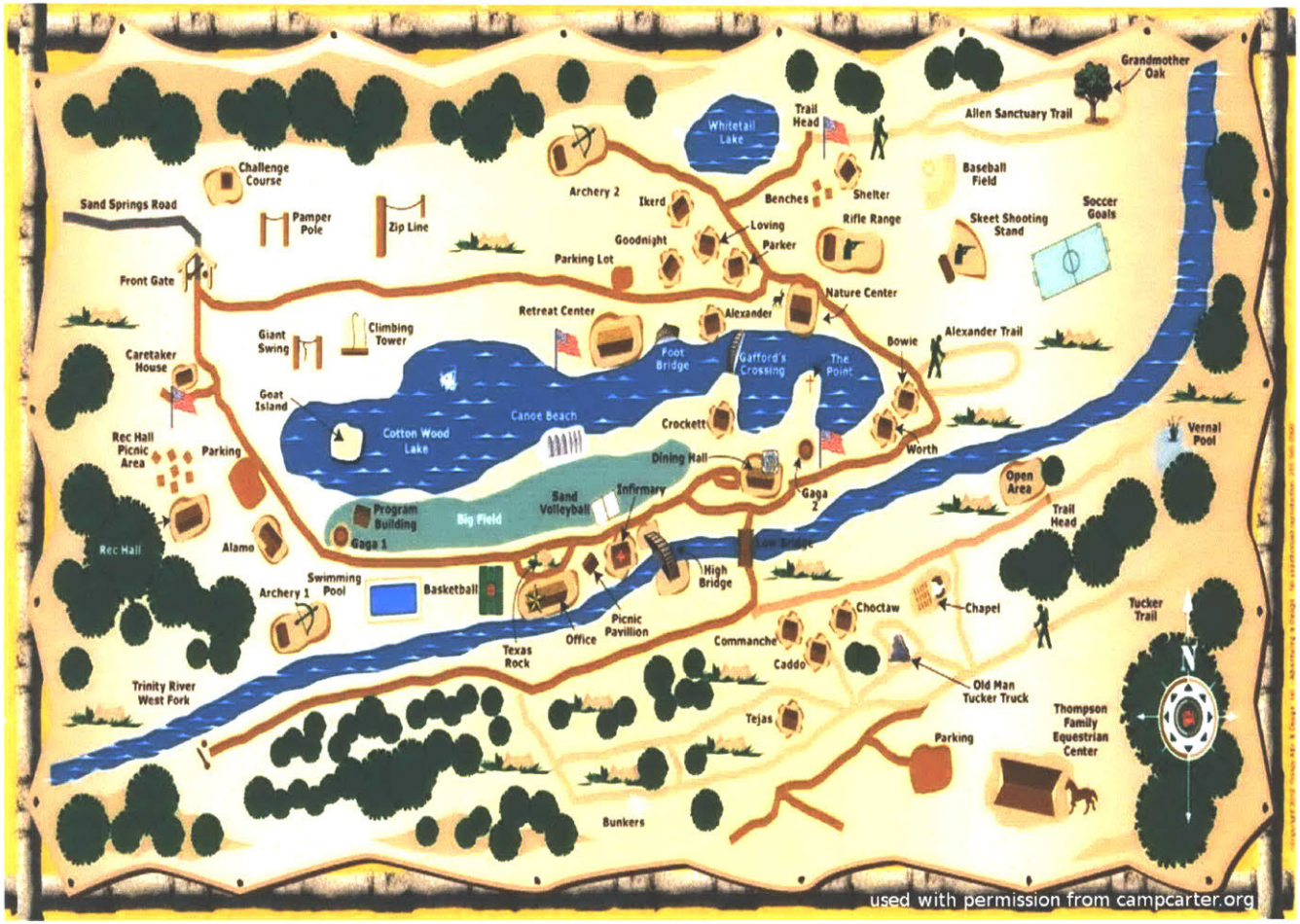


Figure 4-4: Blank camp map given to annotators to produce lost-person scenarios

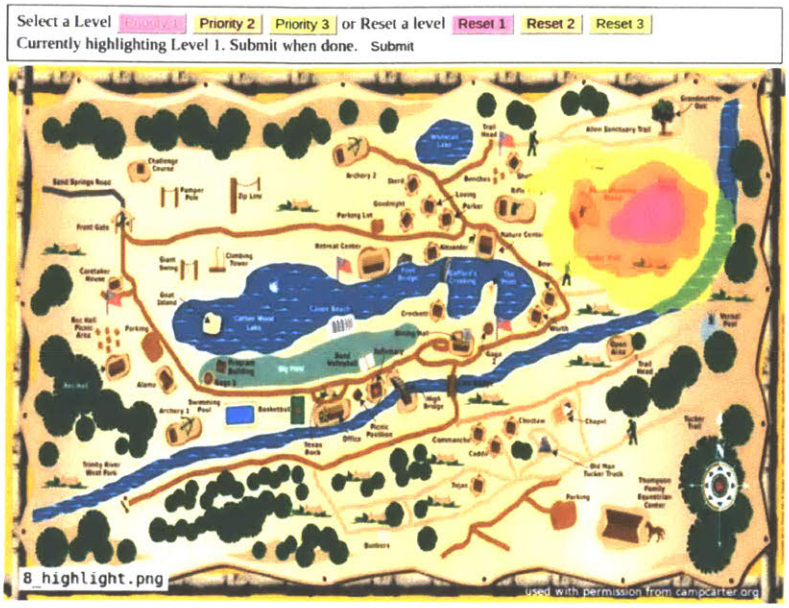


Figure 4-5: *Data collection interface* and annotator response for where to search in response to prompt: "Our best guess of where to search is the area by the Soccer Goals and the Trinity River. The camper is an avid soccer player and never misses a chance to try out a field. The camper said he was bringing his soccer ball to the park today. The camper was last seen a half hour ago walking east past the Skeet Shooting Stand." This highlighting task is the second of three Amazon Mechanical Turk tasks, with the text prompt generated by AMT annotators during a previous round of data collection.

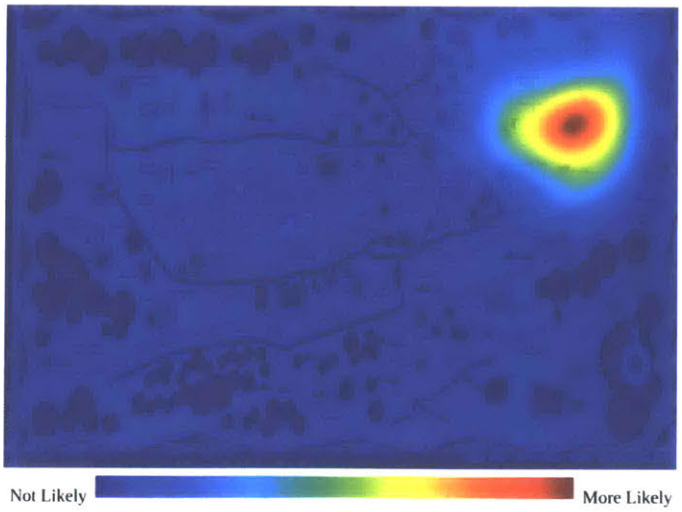


Figure 4-6: *Unnormalized probability map* resulting from combining 5 responses like those in Figure 4-5 through a weighted sum and applying a Gaussian filter. This map was presented to annotators in a third round of AMT data collection to obtain multiple instructions including "Search near the Soccer Goals" and "Search the soccer field to the east, as well as the Alexander Trail and skeet shooting range to the south and west."

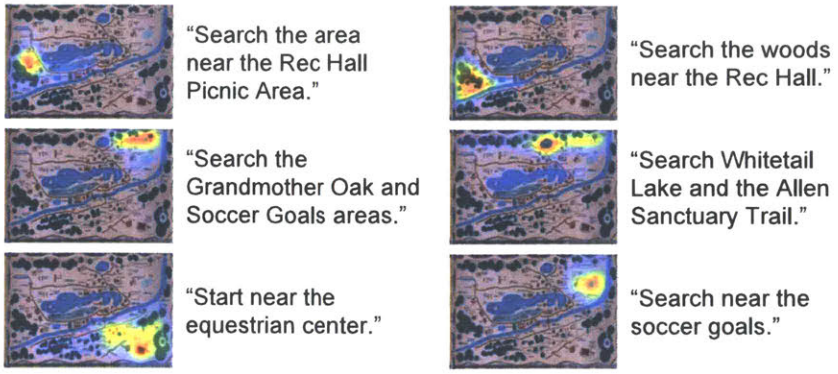


Figure 4-7: Samples from the corpus

commands. The other set of examples consists only of landmarks, to learn the association between landmarks and the text of their names on a map. The model that was learned from the two types of training examples together allows us to ground commands with respect to landmarks in a map.

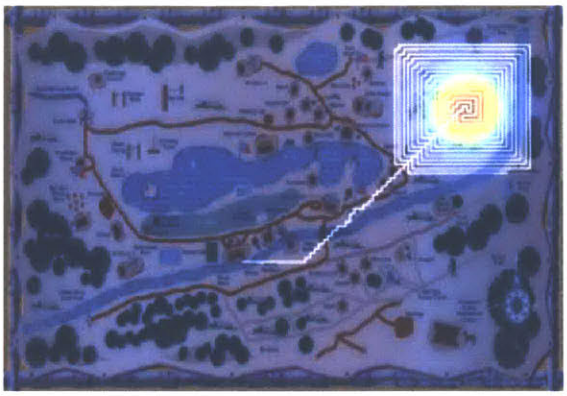
The observation that an expert can hand-select a small subset of the data to train a model that performs well on the whole dataset motivated my application of active learning to this problem (see Chapter 5), so that the subset of data to be labeled can be automatically selected.

4.6 Evaluation and Results

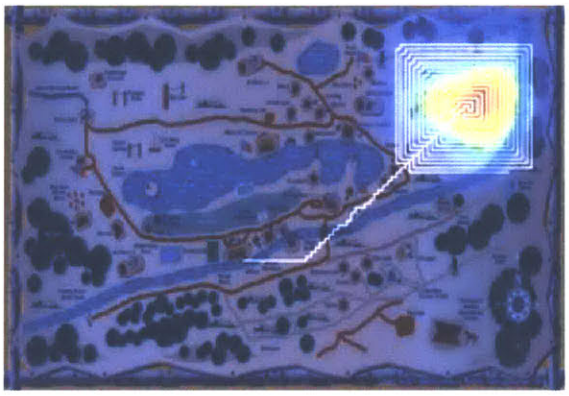
In this section, trajectories generated from natural language commands via an inferred belief represented as grounded basis functions are presented and analyzed. Quantitatively, the probability of success of these trajectories is compared with the probability of success of four other types of trajectories both on individual examples and in summary over 127 language-belief pairings obtained from human annotators.

4.6.1 Quantitative Results

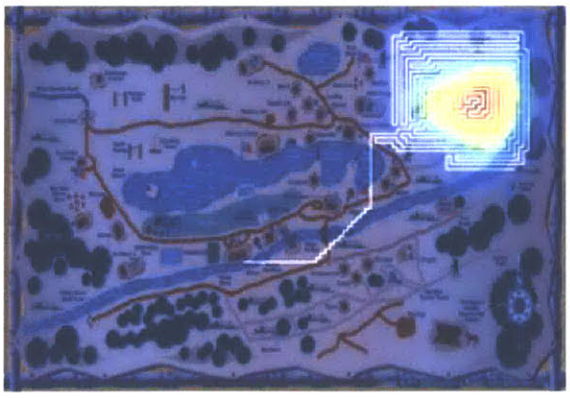
During successful execution, DCG infers a set of active basis functions from a natural language command. These basis functions are combined into a single belief, which



(a) A trajectory planned on the inferred belief and displayed over the inferred belief grounded from a command



(b) A trajectory planned on the inferred belief and displayed over the human-annotated PDF for the corresponding scenario



(c) A trajectory planned on the human-annotated PDF and displayed over the human-annotated PDF for the scenario

Figure 4-8: *Output and qualitative comparison for the command* "Search near the soccer goal". Figures 4-8a and 4-8b show the trajectory generated by planning on the inferred belief overlaid on the inferred belief and human-annotated PDF, respectively. The trajectory planned on the inferred belief appears qualitatively reasonable over the human-annotated PDF. Figure 4-8c shows a trajectory planned on the human-annotated PDF for the scenario. The trajectories planned on the two distributions are qualitatively similar and have a comparable probability of success evaluated on the human-annotated PDF. All three trajectories shown were planned with a T=1000 step limit.

is then used to plan a search trajectory. The resulting belief and trajectory for the instruction "search near the soccer goals" is shown in Figure 4-8a.

Of the 240 simple instructions collected, H2SL successfully grounded 127. Some of the remaining instructions had parse structures that are not reflected in the grammar, while other instructions were parsed correctly but could not be grounded successfully due to limitations in the implemented features.

To quantitatively evaluate this method on the 127 examples for which trajectories could be generated, execution of the output trajectory on the human-annotated PDF associated with the command is simulated and the probability of success of this trajectory is estimated(Figure 4-8b). Since the behaviors of the trajectories may vary with the maximum number of steps T, trajectories are planned over a range of T in 10-step increments and evaluate the probability of success versus T.

The comparison considers the probability of success evaluated on the human-annotated PDF of the following trajectories as functions of the maximum number of steps, T:

1. τ_{basis}, the trajectory planned on the inferred belief.

2. $\tau_{\text{annotated}}$, the trajectory planned on the human-annotated PDF.

3. τ_{sweep}, a parallel sweep pattern over the minimum bounding rectangle containing the non-zero probability cells of the inferred belief.

4. τ_{prior}, the trajectory planned on the prior distribution of missing persons over a map.

5. τ_{bound}, the potentially infeasible trajectory that visits the T most probable cells in the map in decreasing order of probability in T time steps by ignoring travel costs between cells.

The trajectory τ_{basis} results from the natural language command and is the primary trajectory of interest. If the inferred belief represents the human-annotated PDF well, the probability of success of τ_{basis} and $\tau_{\text{annotated}}$ should be similar.

The probability of success of τ_{sweep} and τ_{prior} provide lower bounds on expected performance of this system. Since a parallel sweep efficiently covers a rectangular region, it is a reasonable trajectory when treating the DCG symbolic groundings as specifying whether a grounded region should be searched without considering belief within those regions. The prior was generated without any language input by averaging the human-annotated PDFs of the 80 scenarios collected in the user study. A trajectory planned with language input grounded to basis functions should do at least as well as τ_{sweep} to justify the use of basis functions and at least as well as τ_{prior} to justify the use of language grounding at all.

The probability of success of τ_{bound} provides an absolute upper bound on the performance of this system. Since the planner is approximate, there may be some cases where the probability of success of the trajectory generated on the human-annotated PDF is artificially low, and thus it should not be used alone as an upper bound. However, even the optimal feasible trajectory of an exact planner cannot perform better than τ_{bound}, which is not limited by the physical constraints of motion, so it is included as a means of evaluating the trajectories of the greedy planner.

Figure 4-9 shows the probability of success versus search duration T for a single language-belief pair with command "Search near the soccer goals". A greedy planner works particularly well in this case because the corresponding belief (Figure 4-6) is relatively simple. Thus, the probability of success of $\tau_{\text{annotated}}$ closely follows τ_{bound} with an offset for the number of steps needed to reach the high probability region. The probability of success of τ_{basis} closely matches that of $\tau_{\text{annotated}}$, indicating a good representation of the highest-probability portion of the belief. The parallel sweep pattern is unable to prioritize high-probability regions over low probability regions, and so performs worse than τ_{basis} in shorter time frames, but is able to search the full space covered by τ_{basis} and with sufficient time. Searching purely based on the prior has variable success as the subregions of the prior that are most promising for a given T may or may not overlap with the desired search region based on the language.

In order to summarize the performance across 127 language-belief pairings, results where the probability of success and search times have been non-dimensionalized are

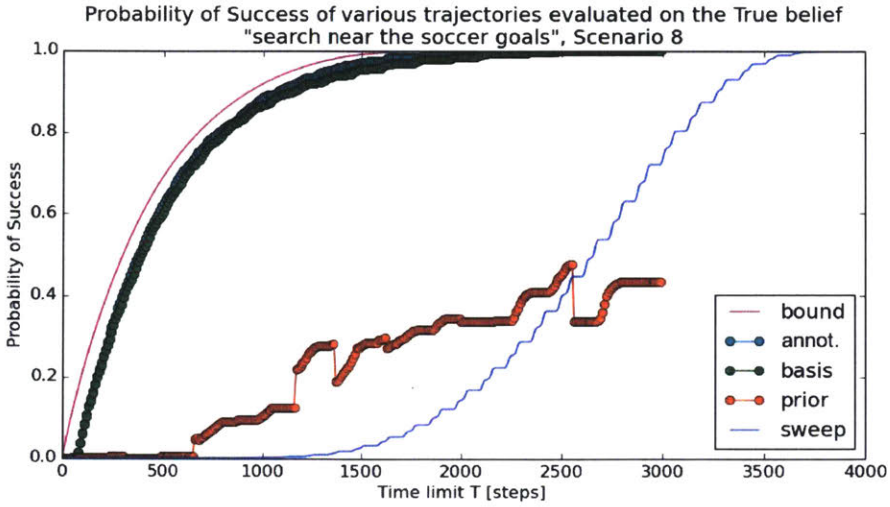


Figure 4-9: *Probability of successfully finding the target for various search durations* T, in steps, in response to the command "search near the soccer goals". The curve for this method (basis) nearly overlaps the plan computed using the true distribution supplied by humans. The coverage strategy (sweep) eventually reaches a high probability of success, but takes nearly ten times as long as this method.

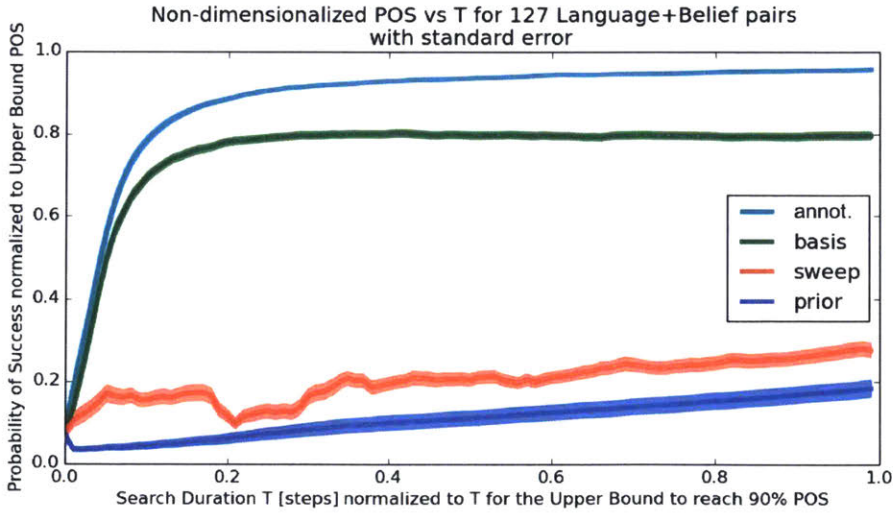


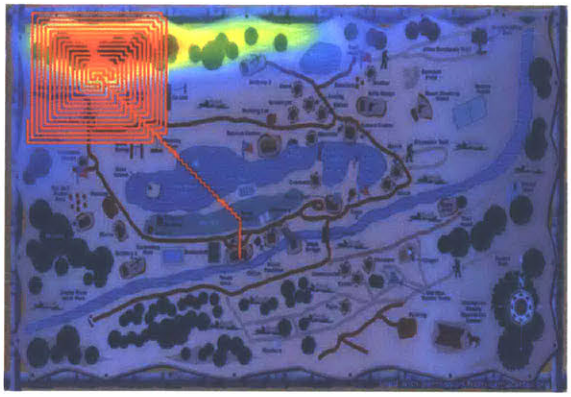
Figure 4-10: *Performance summary of the 127 language commands* that were successfully grounded. The probability of success is normalized relative to the upper bound, and the abscissa is normalized so that $T = 1$ corresponds to the upper bound on search performance reaching a probability of success of 90%.

Figure 4-11: Comparison of the probability of success of various approaches

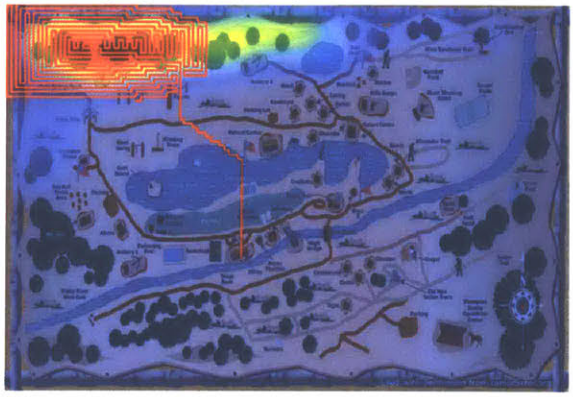
also reported. To remove dependence of probability accumulation rate on the distance of the non-zero probability regions from the start location, the minimum number of steps before any feasible trajectory (any trajectory aside from τ_{bound}) reaches a non-zero probability cell is subtracted from the time. The horizontal axis is then scaled by the number of steps required for τ_{bound} to reach 90% probability of success. The probability of success values are also scaled by the upper bound on the probability of success, yielding a measure of the efficiency of the trajectory.

Plotting the mean and standard error of these normalized quantities in Figure 4-10 reveals several trends. The greedy planner generally performs well since the average efficiency of $\tau_{\text{annotated}}$ compared to the upper bound is typically around 90%. The trajectories planned on the inferred belief are nearly as efficient as $\tau_{\text{annotated}}$ for short search durations, with efficiency during longer searches slightly less at around 80%. This result reflects a good representation of the higher probability regions that are searched first, but less accuracy in representing the low probability regions that should be searched with more available time. Regions of the human-annotated PDF that were not included in the human's command contribute to the gap in efficiency. Planning on the inferred belief still performs much better than planning on the prior, which generally has low success, or executing a parallel sweep over the bounding box of the non-zero probability region, which is slow at accumulating probability since the edges of the box are low probability regions.

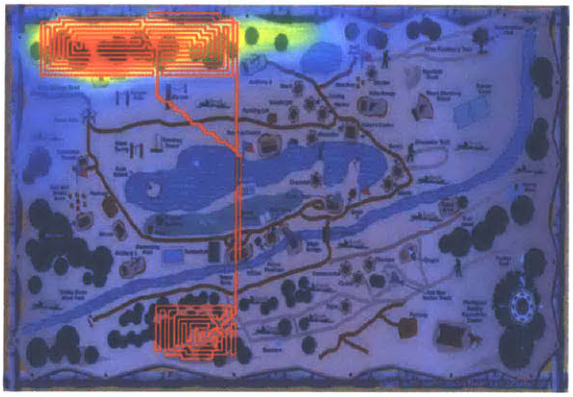
Note that the average performances of τ_{basis} and τ_{sweep} have a dependence on the variance of the Gaussian smoothing used to define the bases. With a small variance on the Gaussian smoothing, the basis can underestimate the search region such that the planner prematurely terminates search, leading to decreasing efficiency over longer searches. In contrast, the parallel sweep performs better when the inferred belief's bounding box underestimates the search region because it can immediately focus on higher probability regions. While a large variance on the Gaussian smoothing allows the planner to make use of additional time, too large a variance can obscure the structure of the search region, and does not fully close the efficiency gap because it does not account for probability mass in distant regions not described by the human.



(a) Trajectory for "search near the challenge course" shown over the human-annotated PDF. Language provided by an AMT user.



(b) Trajectory for "search the woods near the challenge course" shown over the human-annotated PDF. Language provided by an AMT user.



(c) Trajectory for "search the woods" over the human-annotated PDF. Language selected for comparison.

Figure 4-12: Trajectories generated in response to different commands indicating that the woods in the northwest corner of the map should be searched. The ability to ground "the woods north of the Challenge Course" allows the search to be concentrated over the woods, rather than relative to the challenge course. Searching "the woods" without context will have variable results in different scenarios.

4.6.2 Qualitative Results

This work is able to provide reasonable search trajectories for basic commands (e.g. "Search Whitetail Lake") and language commands including composition (e.g. "Search near Allen Sanctuary Trail and the Grandmother Oak") and hierarchy (e.g. "Search south of the Big Field").

Grounding language with these structures allows the representation of beliefs that are difficult to represent with simpler language models. Figure 4-12 shows three trajectories generated from different language commands directing the robot to search the woods in the northwest. Grounding "search the woods near the Challenge Course" results in better trajectories over the human-annotated PDF than grounding commands with simpler structure. The system can also provide reasonable trajectories when multiple areas to search are not adjacent, as for "Search Archery 1 and Archery 2", shown in Figure 4-14a.

However, there are cases that are not currently supported by the system. The system cannot infer parts of the belief that are not specified in the language, potentially leaving important regions unsearched. For example, the command "Search Archery 1 and Archery 2" was given as a concise command for the belief shown in Figure 4-14b. While this command and resulting trajectory in Figure 4-14a capture a significant part of the probability of finding the person, neither accounts for the probability along the road connecting the archery ranges. Grounding a detailed command for this scenario, such as "Search Archery 1 and Archery 2 and the road between them", could activate a basis function over the road, but enabling DCG to ground commands of this complexity is an area for future work.

4.7 Summary

This chapter described an extension to the probabilistic grounding graph methods for natural language interaction with robots to allow the planning of trajectories for outdoor search and rescue. This work demonstrates a system for planning search trajectories from natural language instructions by introducing an intermediate step

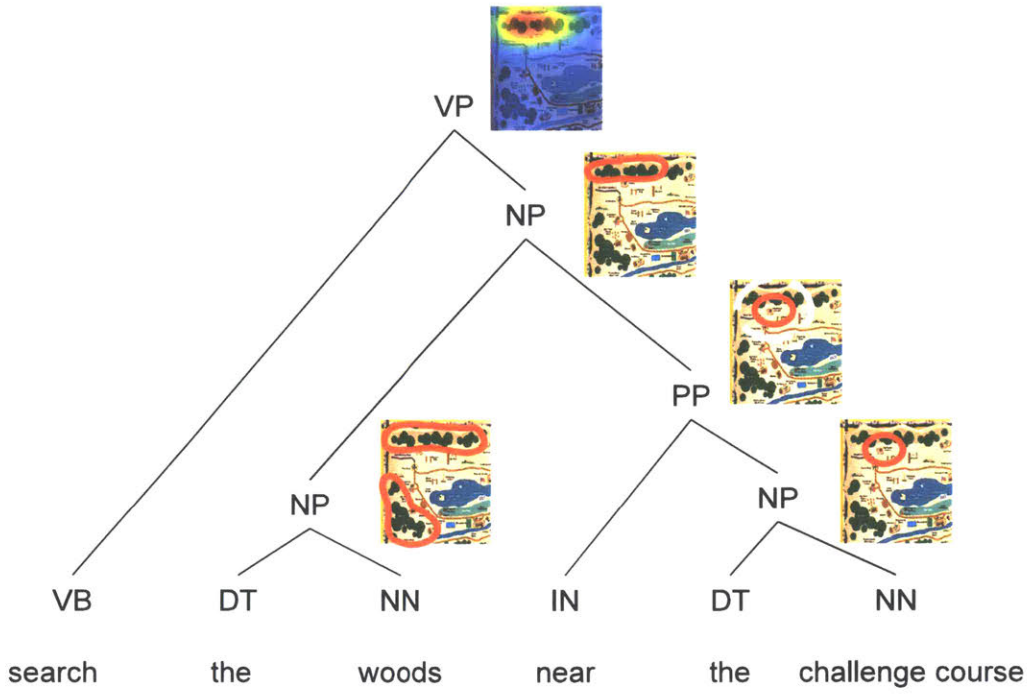
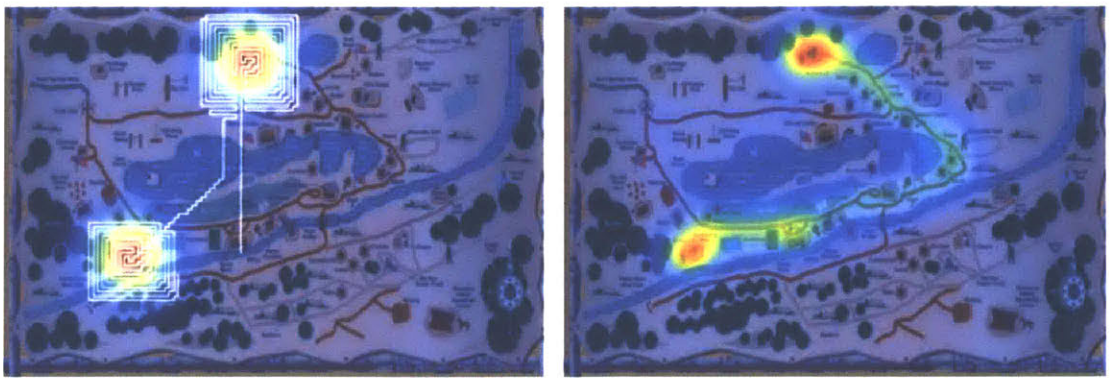


Figure 4-13: Pictorial representation of intermediate groundings at factors along the parse tree for the command "search the woods near the challenge course".



(a) Trajectory for "search archery 1 and archery 2" over the inferred belief

(b) human-annotated PDF for the scenario also includes probability over the road between the archery ranges.

Figure 4-14: The system can provide trajectories when search regions are not adjacent (Figure 4-14a) but does not know to search areas not specified by the language (Figure 4-14b).

that grounds a natural language command to the activation of basis functions that combine to represent a belief of a missing person's location. This system planned 127 trajectories in response to natural language commands and evaluated them on a data set of belief and command pairs generated by annotators on Amazon Mechanical Turk. The resulting trajectories are observed in simulation to have a high probability of success compared to other trajectory types, particularly with shorter time limits on the duration of search.

This work would be improved by extensions in both its basis function representation and language understanding capabilities. Learning a mapping between language, geographic features, and basis function parameters rather than using a fixed function may enable more accurate belief representations, while the ability to ground more complex commands would enable inference of more complex beliefs.

While this work demonstrates utility in grounding language to basis functions for search and rescue, this approach may also be useful in other problem spaces where the cost function can be composed of a finite number of bases.

Chapter 5

Active Learning with Grounding Graphs

Active learning is a well-studied approach to reduce the burden of data annotation by allowing a learner to select examples for labeling based on heuristics for which labels will be most informative (see Section 2.3). An efficient active learning algorithm requests labels for examples that provide new information and avoids labeling redundant examples. This section describes an initial approach to incorporating active learning with probabilistic grounding graphs.

5.1 The Active Learning Algorithm

The generic active learning algorithm repeats the following steps: training on the available data, evaluating unlabeled data based on the trained model to determine the next most informative examples, and adding them to the training pool. These steps are shown in Figure 5-1. As noted in Section 2.3, many heuristics have been used to select the examples to label next.

Entropy is commonly used as a measure of uncertainty to determine the most informative samples. However, in the formulation of probabilistic grounding graphs, the probability $p(\gamma_i|\lambda_i, \phi_i, \Gamma_{c_i}, \mathcal{M})$ that would be used to compute the entropy for a grounding variable is not readily available. While Tellex et al. [42] formally approxi-

```
Initially:
    0. Label an initial set from the pool.
Repeat Steps 1-3:
    1. Train a model with all available labeled data.
    2. Using the trained model, evaluate the remaining examples
       in the pool. Select unlabeled examples for labeling
       according to active learning heuristic.
    3. Label selected examples.
```

Figure 5-1: *Generic Pool-Based Active Learning Cycle.* Pool-based active learning assumes access to a large pool of unlabeled data at low cost, and the ability to label data from that pool at a higher cost. Active learning heuristics that can be used in Step 2 to determine which examples to label next include measures of disagreement with the existing model, expected entropy reduction for receiving a label, etc.

mate the entropy using the probabilities of the candidate groundings that populate the beam during inference, they note that the correspondence probability $p(\phi_i|\gamma_i, \lambda_i, \Gamma_{c_i}, \mathcal{M})$ is readily available and may also serve as a measure of uncertainty. Though there are cases in which a high correspondence probability will occur even with high entropy on the grounding variable (e.g., two candidate groundings are each well-suited and equally likely), the correspondence probability may still be used to generate informative heuristics for active learning.

5.2 Experimental Setup

The experiments in this work compare the *learning curves* when selecting data with four active learning heuristics and selecting data randomly. A learning curve shows the number of correct predictions as a function of the number of examples used in training. More effective data selection methods result in higher prediction accuracy with fewer training examples. For each learning curve, the model is first trained on the same initial selection of data. Subsequent labeled examples are requested by the learner.

5.2.1 Heuristics

In this work, I explore four different active learning heuristics all based on the correspondence probability $p(\phi_i|\gamma_i, \lambda_i, \Gamma_{c_i}, \mathcal{M})$. As noted in Chapter 3, each example gives the grounding for a full language instruction. The instruction can be decomposed into multiple Phrases according to the parse tree, resulting in multiple smaller grounding problems. Each training example can be treated as a single unit (in which all Phrases in the parse tree are labeled at once), or can be treated as a source of multiple examples at the Phrase level.

Similarly, the probability of the groundings can be computed for an entire parse tree ($p(\Phi|\Gamma, \Lambda, \mathcal{M})$) or can be computed at the Phrase level ($p(\phi_i|\gamma_i, \lambda_i, \Gamma_{c_i}, \mathcal{M})$). Recalling that conditional independence assumptions allow the factorization

$$p(\Phi|\Gamma, \Lambda, \mathcal{M}) = \prod_{i=1}^{|\Lambda|} p(\phi_i|\gamma_i, \lambda_i, \Gamma_{c_i}, \mathcal{M}), \tag{5.1}$$

where Γ_{c_i} are the groundings of the child phrases of the language phrase λ_i, probabilities of subtrees can also be computed as the product of Phrase probabilities in that subtree. I explore active learning heuristics at each of these resolutions in my experiments. The four heuristics explored are as follows:

- **Heuristic 1: ActiveRootTree** The correspondence probability of the entire tree is computed at the *root*. All Phrases (the entire *tree*) in the example with the lowest probability at the root are labeled and added to the training data.
- **Heuristic 2: ActiveSubtreeTree**. The correspondence probability of the *subtree* rooted at each Phrase is computed. All Phrases (the entire *tree*) in the example with the lowest probability at the subtree level are labeled and added to the training data.
- **Heuristic 3: ActivePhraseTree**. The correspondence probability of each *Phrase* is computed. All Phrases (the entire *tree*) in the example with the lowest probability at the Phrase level are labeled and added to the training data.

- **Heuristic 4: ActivePhrasePhrase**. The correspondence probability of each *Phrase* is computed. Only the *Phrase* with the lowest probability at the Phrase level (and not all Phrases in the parse tree) is labeled and added to the training data.

The performance of these heuristics is additionally compared to the performance of random selection. In the random selection case, all Phrases in a randomly chosen example are labeled at once.

5.2.2 Data Set

The data set used for this experiment is the training set provided with the H2SL software[1] accessed in April 2016. The data set consists of 32 examples with language instructions for navigation to be performed in a toy home environment with objects including chairs, boxes, and tables. The 32 examples contain total 137 Phrases, with two to six Phrases per example. The full list of instructions can be found in the Appendix in Section A.6. The first 9 examples (33 Phrases) were arbitrarily chosen for inclusion in the initial training set.

5.3 Results

The results of the active learning experiment is shown in Figure 5-2, where the learning curves for the four active learning heuristics are compared against a random sampling learning curve. The random sampling learning curve represents the mean with standard error for 10 runs. The horizontal axis of the fourth active learning heuristic (ActivePhrasePhrase), which labels new data one Phrase at a time, is scaled to the average of 4.52 phrases per example in the unlabeled pool for comparison with the other heuristics that label a full example at once.

The first three active learning heuristics that label all phrases from an instruction at once (ActiveRootTree, ActiveSubtreeTree, and PhraseTree) perform similarly, and,

[1] https://github.com/tmhoward/h2sl

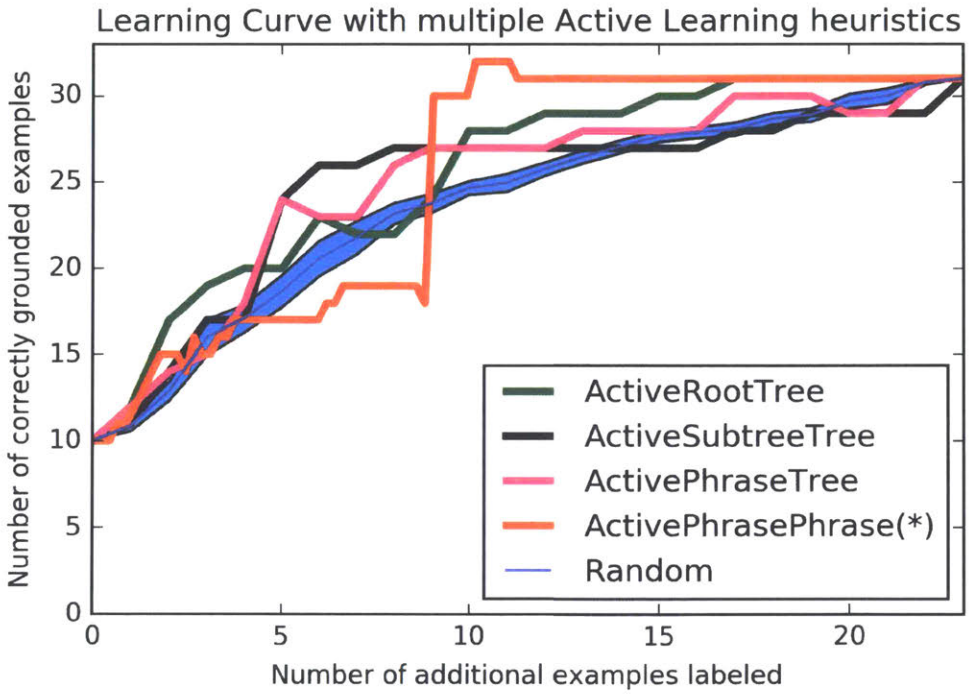


Figure 5-2: *Active Learning Results.* This figure shows the learning curves of 4 active learning heuristics and the mean and standard error for 10 runs of random sampling. The first three active learning heuristics (ActiveRootTree, ActiveSubtreeTree, and ActivePhraseTree) and the random sampling method (Random) label one full example containing multiple Phrases at each selection step. The fourth active learning heuristic (ActivePhrasePhrase) labels one Phrase at a time, and its horizontal axis has been scaled by the average number of Phrases per example. The first three active learning heuristics perform similarly and generally dominate the mean random performance. ActivePhrasePhrase has more interesting behavior. Its initial underperformance occurs when labeling Phrases that have low certainty resulting from a lack of distinguishing characteristics of the Phrase. Its performance drastically improves when it labels new types of Phrases and reaches full accuracy with significantly fewer total labeled Phrases than the other methods.

with a few exceptions, dominate the performance of random sampling.

The fourth active learning heuristic (ActivePhrasePhrase) that labels individual Phrases rather than all Phrases from an example at once, has a different behavior. This heuristic underperforms initially, then drastically improves to dominate all other methods, achieving full accuracy with labels for approximately two-thirds of the total number of available Phrases. Upon inspection of the intermediate steps, it was determined that many of the Phrases selected for labeling early in the process were rated with low confidence due to a lack of distinguishing features. Labeling these examples provided little additional information to the learner. The performance significantly improved when the learner asked for the meaning of a verb that was not contained in the initial labeled set. The Phrases whose labels were requested last contained verbs that were prevalent in the initial training set, for which the learner was already confident. This behavior that prioritizes new labels over familiar ones allows fewer total labels to be needed to reach high accuracy.

5.4 Summary

This chapter shows the application of a simple pool-based active learning algorithm to grounding natural language instructions with probabilistic grounding graphs. The exploration in this work shows a potential for savings in annotation through the use of active learning. The results show a minor improvement for three simple active learning heuristics that label the groundings for full instructions, and interesting behavior of a fourth heuristic that labels groundings for Phrases within an instruction. Labeling individual Phrases within examples allowed fewer total Phrases to be labelled, since elements unique to certain instructions could be labeled while elements common to multiple instructions could be skipped. It should be noted, however, that labeling at the Phrase level might be challenging for the expert annotator in practice. In this experiment, the learner could query the groundings for specific Phrases from a database containing the full groundings of all Phrases in each example. An expert human annotator may find it difficult to specify groundings for Phrases higher in the

parse tree without also considering the groundings for Phrases lower in the parse tree, reducing some of the benefits. Future work could consider entropy-based and other heuristics as well as practical considerations of partial labeling. This work should also be repeated on larger, less curated data sets, to see whether benefits of active learning become more pronounced when the data is more repetitive.

Chapter 6

Conclusion

The first part of this thesis addressed the issue of enabling natural language interaction with micro aerial vehicles for outdoor search and rescue. Micro aerial vehicles extend the range of outdoor search, while natural language interfaces could allow human volunteers to interact with the vehicles with minimal training. The approach presented in this work extended existing work in grounding natural language commands using probabilistic grounding graph methods. In particular, I proposed a new type of grounding, a basis function that represents a contribution to a probability density function. The representations in previous works that grounded natural language to trajectories or constraints were ill-suited to capturing the form of search trajectories. The new basis function groundings allowed approximations of probability density functions, or beliefs, of a missing person's location to be inferred from language. Given these beliefs, search trajectory planning methods can be used to generate paths. Experimental results in simulation showed that trajectories planned on the beliefs inferred from language performed well when compared to trajectories planned on human-annotated beliefs, particularly for short search durations. Future work in this area would include support for a greater variety of language expressions and incorporation of spatial understanding to the basis functions.

The second part of this thesis addressed the challenge of acquiring sufficient labeled training data to learn models in new environments. Annotating natural language structures with corresponding groundings as used in previous probabilistic ground-

ing graph approaches is time-consuming. Data sets often include redundant labels that provide little new information. I applied pool-based active learning to actively select subsequent examples for labeling based on multiple uncertainty heuristics, and demonstrated the potential for active learning to reduce the amount of labeling effort require to attain similar performance. Future work in this area would include considering entropy-based heuristics, feasibility of providing partial labels, and applications to larger data sets.

Appendix A

Supplementary Implementation Material

This section contains sample data files and code used in H2SL and pyh2sl implementations. A more high-level overview of the implementation can be found in Section 3.2.

A.1 The Grammar

A grammar is a list of production rules that properly constructed text must obey. Figure A-1 shows a simple grammar that could be used in a toy home environment. See Section 3.2.2 for more information.

```
1  S -> NP VP | VP | NP | PP
2  VP -> VB PP | VB NP | VP CC VP | VB
3  PP -> IN NP | TO NP
4  NP -> NP PP | DT NN | JJ NN | NP CC NP | NN
5  CC -> "and"
6  DT -> "the"
7  IN -> "near" | "toward" | "towards" | "by" | "of" | "behind"
8  NN -> "box" | "crate" | "package" | "table" | "desk" | "chair" | "
       sofa" | "sink" | "fridge" | "refrigerator" | "hammer" | "mallet"
       | "pipe" | "left" | "right" | "front" | "back" | "bottle" | "seat
       " | "stool" | "counter"
9  TO -> "to"
10 VB -> "walk" | "drive" | "move" | "go" | "approach" | "leave" | "
       exit" | "proceed"
```

Figure A-1: **A simple grammar showing production rules.** Beginning with an input string S, elements on the left of the arrows can produce any of their following elements on the right of the arrows, with options separated by the vertical bar. Aside from the initial string S, elements on the left include phrases (verb, prepositional, and noun phrase types denoted as *VP*, *PP*, and *NP*, respectively) and unary-production pre-terminals (nouns, verbs, etc.) that produce terminal words, indicated by quotation marks. A parse tree specifies the branching path through the set of productions that goes from the input string S to the words in that string.

A.2 The Parse Tree

The Natural Language Toolkit (NLTK) in Python has built-in parsers that can be used to generate and display parse trees for input **text** given a **grammar**. The code in Figure A-2 will parse the string "go to the box" and display a parse tree. See Section 3.2.2 for more information.

```
1  import nltk
2
3  def nltk_parse(text, grammar_file, verbose=False):
4      '''
5      This function takes in text (e.g. a phrase or sentence) and
       calls
6      standard nltk tools to obtain a list of possible parse trees.
7
8      '''
9      grammar = nltk.data.load(grammar_file)
10     sent = text.split()
11
12     rd_parser = nltk.ChartParser(grammar)
13     tree_list = list(rd_parser.parse(sent))
14
15     if len(tree_list) == 0:
16         sys.exit("No parses found")
17
18     if verbose:
19         for tree in tree_list:
20             tree.draw()
21
22     return tree_list
23
24 if __name__ == "__main__":
25     nltk_parse("go to the box", 'file:grammar.cfg', verbose=True)
```

Figure A-2: Using a Natural Language Toolkit parser

A.3 The Factor Graph Representation

The Phrase class shown in Figure A-3 is the primary structure for representing the factor graph. See Section 3.2.3 for more information.

```
class Phrase(object):
    """
    Class to represent a phrase.
    type: phrase type (NP, VP, etc.).
    words: list of tokens representing words of the phrase
    children: ordered list of child phrases
    grounding: grounding of current phrase. Instance of class
    Grounding.
    """
    def __init__(self):
        self.type = None
        self.words = []
        self.children = []
        self.grounding = None
        self.beam = None
```

Figure A-3: The Phrase class

A.3.1 pyh2sl Groundings

The pyh2sl grounding structure was motivated by a need for a flexible grounding structure that could grow with a language description. In my pyh2sl implementation, I define a Grounding object with a relation, a list of child groundings, and a dictionary of properties. Groundings in the list of child groundings may have their own child groundings, allowing for nested symbolic groundings that can parallel nested descriptions in language. In contrast to the H2SL groundings (see Section 3.2.4), the search space for these groundings cannot be fully enumerated, so the grounding procedure has a slight modification, as described in Section 3.2.6. The pyh2sl grounding structure has worked well for basic instructions. Thorough testing with more complex

instructions remains as future work. The pyh2sl Grounding class is shown in Figure A-4.

```
class Grounding(object):
    """
    Class to store groundings.
    .relation refers to e.g. union, intersection, object, region
    types, constraint types.
        'object'                          i.e. 'object'
        'region:REGION_TYPE'              e.g. 'region:north'
        'constraint:CONSTRAINT_TYPE'      e.g. 'constraint:inside'
    .groundings is a list of child Groundings
    .properties is a dictionary of properties.  (e.g. object
    properties)  e.g. {'type':'table'}

    "Go to the woods north of the flagpole" -->
    "flagpole": G1: Grounding('object', None, flagpole_properties)
    "north of the flagpole": G2: Grounding('region:north', [G1],
    None)
    "woods north of the flagpole":
    G3: Grounding('intersection', [G1, G2], None)
    "Go to the woods north of the flagpole":
    G4: Grounding('constraint:inside', [G3], None)
    """
    def __init__(self, rel=None, gnd=None, prop=None):
        self.relation = rel
        self.properties = prop if prop is not None else {}
        self.groundings = gnd if gnd is not None else []
```

Figure A-4: The Grounding class

A.3.2 H2SL Features

The features used in H2SL highlight co-occurrences of language and grounding elements. One set of base features is defined to highlight language elements, such as whether a specific word or part of speech is present in a phrase. A second set of base features is defined to highlight grounding elements, such as whether a certain object or relationship is involved in the grounding of a phrase. The actual set of features used in H2SL is the Cartesian product of these two sets of base features.

This section contains lists of language and grounding base features used in H2SL. For a description with examples, see Section 3.2.5.

Language Base Features

Following is a list of language features used in pyh2sl. These features all parallel the features in H2SL. Each feature name is given, followed by the condition under which the feature will compute as true.

- (Multiple Features) Has part of speech
  - The part of speech of the language is a [noun phrase / prepositional phrase / noun / preposition / etc.] from the grammar
- (Multiple Features) Has word
  - The word is ["go" / "to" / "box" / "desk" / etc.] from the grammar

Grounding Base Features

Following is a list of grounding features used in pyh2sl. Nearly all these features parallel the features in H2SL. Each feature name is given, followed by the condition under which the feature will compute as true.

- Grounding matches child grounding
  - The grounding of this phrase is identical to the grounding of the child phrase.

- Region reference matches child grounding
  - The grounding of this phrase is a region whose reference grounding is identical to the grounding of the child phrase.
- Constraint child matches child grounding
  - The grounding of this phrase is a constraint whose second reference is identical to the grounding of the child phrase.
- Constraint child matches child region
  - The grounding of this phrase is a constraint whose second reference is a region that is identical to the grounding of the child phrase.
- Constraint parent matches child region
  - The grounding of this phrase is a constraint whose first reference is a region that is identical to the grounding of the child phrase.
- Constraint parent is robot
  - The grounding of this phrase is a constraint whose first reference is a robot object.
- Grounding references all match child groundings
  - Every grounding in the list of reference groundings of this phrase is a grounding of the child phrase.
- Region merge partially known regions
  - The grounding of this phrase is a region whose type matches the grounding of one child phrase and whose reference matches the grounding of a different child phrase.
- (Multiple Features) Type field of this grounding is X

- The grounding of this phrase is a [constraint / region / region type / object / etc.] whose type is [inside / near / left / box / desk / etc.]

- (Multiple Features) Grounding is a grounding of type X

  - The grounding of this phrase is a [constraint / region / region type / object / etc.]

A.4 Training Files

Training examples are specified in an XML format. Figure A-5 is an excerpt of a training file showing the natural language instruction and the world model specification. Figure A-6 shows the grounding for the top level verb phrase. The groundings for the lower level phrases are not shown.

```
<?xml version="1.0" encoding="UTF-8"?>
<root>
   <instruction text="go to the box"/>
   <world time="0">
     <object name="o0" type="robot"/>
     <object name="o1" type="box"/>
     <object name="o2" type="desk"/>
     <object name="o3" type="chair"/>
   </world>
   ⋮
</root>
```

Figure A-5: Sample world model in training data

```
<VP>
  <grounding>
    <grounding_set>
      <constraint type="inside">
        <parent>
          <region type="na">
            <object name="o0" type="robot"/>
          </region>
        </parent>
        <child>
          <region type="na">
            <object name="o1" type="box"/>
          </region>
        </child>
      </constraint>
    </grounding_set>
  </grounding>
  <VB text="go"/>
  ⋮
</VP>
```

Figure A-6: Sample expert-labeled verb phrase

A.5 Training the Log-Linear Model

The code in Figure A-7 can be used to train a log-linear model using scikit-learn's Logistic Regression class in Python.

```
from sklearn.linear_model import LogisticRegression

def get_trained_logreg_model(labeled_vectors):
    '''
    Manipulate sklearn logistic regression with training vectors

    labeled_vectors are a list of (label, vector) pairs.
    '''
    all_labels = [i for (i, j) in labeled_vectors]
    all_data = [j for (i, j) in labeled_vectors]

    logreg = LogisticRegression(solver='lbfgs')
    logreg.class_weight = 'balanced'

    logreg.fit(all_data, all_labels)

    return logreg
```

Figure A-7: *Training the log-linear model* using scikit-learn's Logistic Regression class

A.6 H2SL Toy World Instructions

Figure A-8 shows the navigation instructions used in the active learning experiments in Chapter 5.

```
1  walk to the box
2  walk to the left of the sink
3  walk to the front of the chair
4  walk to the chair and the crate
5  walk to the chair
6  walk to the table
7  walk near the table
8  leave the table
9  exit the chair
10 exit the right of the refrigerator
11 move towards the hammer
12 go by the mallet
13 go toward the pipe
14 drive toward the bottle
15 walk to the front of the fridge
16 go to the crate
17 go to the package
18 proceed to the package
19 drive to the desk and the seat
20 walk to the counter and the stool
21 walk to the left of the crate
22 walk behind the table
23 walk to the back of the table
24 approach the front of the desk
25 go to the front of the desk
26 drive to the right of the sofa
27 drive behind the sofa
28 walk to the right of the desk
29 walk to the desk and the sofa
30 walk to the left of the sofa
31 approach the front of the sofa
32 approach the right of the sink
```

Figure A-8: *H2SL Toy World Instructions.* These 32 navigation instructions are given as sample training data with the H2SL software.

Bibliography

[1] Shilpa Arora and Sachin Agarwal. Literature review: Active learning for natural language processing, 2007.

[2] A. Boularias, F. Duvallet, J. Oh, and A. Stentz. Grounding spatial relations for outdoor robot navigation. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1976–1982, May 2015.

[3] Frédéric Bourgault, Tomonari Furukawa, and Hugh F. Durrant-Whyte. *Optimal Search for a Lost Target in a Bayesian World*, pages 209–222. Springer Berlin Heidelberg, Berlin, Heidelberg, 2006.

[4] David L. Chen and Raymond J. Mooney. Learning to interpret natural language navigation instructions from observations. In *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence*, volume 1, January 2011.

[5] Istvan Chung, Oron Propp, Matthew R. Walter, and Thomas M. Howard. On the performance of hierarchical distributed correspondence graphs for efficient symbol grounding of robot instructions. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5247–5252, September 2015.

[6] David A. Cohn, Zoubin Ghahramani, and Michael I. Jordan. Active learning with statistical models. *CoRR*, cs.AI/9603104, 1996.

[7] Michael Collins. Log-linear models.

[8] Michael Goodrich, Bryan Morse, Damon Gerhardt, Joseph L. Cooper, Morgan Quigley, Julie A. Adams, and Curtis Humphrey. Supporting wilderness search and rescue using a camera-equipped mini uav: Research articles. 25:89 – 110, February 2008.

[9] Nakul Gopalan, Dilip Arumugam, Lawson Wong, and Stefanie Tellex. Sequence-to-sequence language grounding of non-markovian task specifications. In *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, June 2018.

[10] Daniel Gordon, Aniruddha Kembhavi, Mohammad Rastegari, Joseph Redmon, Dieter Fox, and Ali Farhadi. IQA: visual question answering in interactive environments. *CoRR*, abs/1712.03316, 2017.

[11] S. R. Hansen, T. W. McLain, and M. A. Goodrich. Probabilistic searching using a small unmanned aerial vehicle, 2007.

[12] Stevan Harnad. The symbol grounding problem. *Physica D: Nonlinear Phenomena*, 42(1):335 – 346, 1990.

[13] Sachithra Hemachandra, Felix Duvallet, Thomas M. Howard, Nicholas Roy, Anthony Stentz, and Matthew R. Walter. Learning models for following natural language directions in unknown environments. In *Proceedings of the 2015 International Conference on Robotics and Automation*, 2015.

[14] Thomas M. Howard, Istvan Chung, Oron Propp, Matthew R. Walter, and Nicholas Roy. Efficient natural language interfaces for assistive robots. In *Workshop on Rehabilitation and Assistive Robotics at the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014.

[15] Thomas M. Howard, Stefanie Tellex, and Nicholas Roy. A natural language planner interface for mobile manipulators. In *IEEE Intl Conf. on Robotics and Automation (ICRA)*, pages 6652–6659, 2014.

[16] A. S. Huang, S. Tellex, A. Bachrach, T. Kollar, D. Roy, and N Roy. Natural language command of an autonomous micro-air vehicle. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010.

[17] D. Jurafsky and J. Martin. *Speech and Language Processing.* Pearson Education, Inc., Upper Saddle River, NJ, 2009.

[18] Mehran Khodabandeh, Zhiwei Deng, Mostafa S. Ibrahim, Shin'ichi Satoh, and Greg Mori. Active learning for structured prediction from partially labeled data. *CoRR*, abs/1706.02342, 2017.

[19] Robert J. Koester. *: Lost Person Behavior: A Search and Rescue Guide on Where to Look - for Land, Air and Water.* dbS Productions LLC, October 2008.

[20] Thomas Kollar, Stefanie Tellex, Deb Roy, and Nicholas Roy. Toward understanding natural language directions. In *Proceedings of the 5th ACM/IEEE international conference on Human-robot interaction*, pages 259–266, 2010.

[21] David D. Lewis and William A. Gale. A sequential algorithm for training text classifiers. In *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '94, pages 3–12, New York, NY, USA, 1994. Springer-Verlag New York, Inc.

[22] L. Lin and M. A. Goodrich. Uav intelligent path planning for wilderness search and rescue. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 709–714, 10 2009.

[23] L. Lin and M. A. Goodrich. A bayesian approach to modeling lost person behaviors based on terrain features in wilderness search and rescue. 16:300–323, September 2010.

[24] L. Lin and M. A. Goodrich. Hierarchical heuristic search using a gaussian mixture model for uav coverage planning. *IEEE Transactions on Cybernetics*, 44(12):2532–2544, Dec 2014.

[25] L. Lin, M. Roscheck, M. A. Goodrich, and B. S. Morse. Supporting wilderness search and rescue with integrated intelligence: Autonomy and information at the right time and the right place. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence, Special Track on Integrated Intelligence*, July 2010.

[26] James MacGlashan, Monica Babes-Vroman, Marie desJardins, Michael L. Littman, Smaranda Muresan, Shawn Squire, Stefanie Tellex, Dilip Arumugam, and Lei Yang. Grounding English commands to reward functions. In *Robotics: Science and Systems*, July 2015.

[27] Matt MacMahon, Brian Stankiewicz, and Benjamin Kuipers. Walk the talk: Connecting language, knowledge, and action in route instructions. In *National Conference on Artificial Intelligence*, 2006.

[28] C. Matuszek, D. Fox, and K. Koscher. Following directions using statistical machine translation. In *2010 5th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pages 251–258, March 2010.

[29] Robin R. Murphy, Eric Steimle, Chandler Griffin, Charlie Cullins, Mike Hall, and Kevin Pratt. Cooperative use of unmannedsea surface and micro aerialvehicles at hurricane wilma. *Journal of Field Robotics*, 25:164–180, February 2008.

[30] Rohan Paul, Jacob Arkin, Derya Aksaray, Nicholas Roy, and Thomas M. Howard. Efficient grounding of abstract spatial concepts for natural language interaction with robot platforms. *The International Journal of Robotics Research*, June 2018.

[31] Rohan Paul, Jacob Arkin, Nicholas Roy, and Thomas M. Howard. In *Robotics: Science and Systems*, June 2016.

[32] Rohan Paul, Andrei Barbu, Sue Felshin, Boris Katz, and Nicholas Roy. Temporal grounding graphs for language understanding with accrued visual-linguistic context. International Joint Conferences on Artificial Intelligence, 2017.

[33] Nisha Pillai, Karan K. Budhraja, and Cynthia Matuszek. Improving grounded language acquisition efficiency using interactive labeling. In *Robotics: Science and Systems 2nd Workshop on Model Learning for Human-Robot Communication*, June 2016.

[34] Eric Sample, Nisar Ahmed, and Mark Campbell. An experimental evaluation of bayesian soft human sensor fusion in robotic systems. In *AAAI Guidance, Navigation and Control Conference*, August 2012.

[35] Andrew I. Schein and Lyle H. Ungar. Active learning for logistic regression: an evaluation. *Machine Learning*, 68(3):235–265, Oct 2007.

[36] Burr Settles. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison, 2009.

[37] H. S. Seung, M. Opper, and H. Sompolinsky. Query by committee. In *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*, COLT '92, pages 287–294, New York, NY, USA, 1992. ACM.

[38] N. Shimizu and A. Haas. Learning to follow navigational route instructions. In *Proc. 21st International Joint Conference on Artificial Intelligence*, pages 1488–1493, July 2009.

[39] Mohit Shridhar and David Hsu. Grounding spatio-semantic referring expressions for human-robot interaction. *CoRR*, abs/1707.05720, 2017.

[40] Stefanie Tellex. *Natural Language and Spatial Reasoning.* PhD dissertation, Massachusetts Institute of Technology, September 2010.

[41] Stefanie Tellex, Thomas Kollar, Steven Dickerson, Matthew R. Walter, Ashis Gopal Banerjee, Seth Teller, and Nicholas Roy. Understanding natural language commands for robotic navigation and mobile manipulation. In *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence*, AAAI'11, pages 1507–1514. AAAI Press, 2011.

[42] Stefanie Tellex, Pratiksha Thaker, Robin Deits, Thomas Kollar, and Nicholas Roy. Toward information theoretic human-robot dialog. In *Robotics: Science and Systems*, July 2012.

[43] Jesse Thomason, Aishwarya Padmakumar, Jivko Sinapov, Justin Hart, Peter Stone, and Raymond J. Mooney. Opportunistic active learning for grounding natural language descriptions. In Sergey Levine, Vincent Vanhoucke, and Ken Goldberg, editors, *Proceedings of the 1st Annual Conference on Robot Learning*, volume 78 of *Proceedings of Machine Learning Research*, pages 67–76. PMLR, 13–15 Nov 2017.

[44] Cynthia A. Thompson, Mary Elaine Califf, and Raymond J. Mooney. Active learning for natural language parsing and information extraction. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML-99)*, pages 406–414, Bled, Slovenia, June 1999.

[45] N. Trivedi, P. Schermerhorn, P. Langley, and M. Scheutz. Communicating, interpreting, and executing high-level instructions for human-robot interaction. In

Advances in Cognitive Systems: 2011 AAAI Fall Symposium, pages 321–328, 2011.

[46] Mycal Tucker, Derya Aksaray, Rohan Paul, Gregory J Stein, and Nicholas Roy. Learning unknown groundings for natural language interaction with mobile robots. In *International Symposium on Robotics Research (ISRR)*, 2017.

[47] Adam Vogel and Daniel Jurafsky. Learning to follow navigational directions. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics*, pages 806–814, January 2010.

[48] S. Waharte and N. Trigoni. Supporting search and rescue operations with uavs. In *2010 International Conference on Emerging Security Technologies*, pages 142–147, September 2010.

[49] Terry Winograd. Procedures as a representation for data in a computer program for understanding natural language. Artificial Intelligence Technical Report 235, Massachusetts Institute of Technology, January 1971.

[50] WSDOT. *WSDOT Aviation Emergency Services Aircrew Training Reference Text*, chapter 11: Visual Search Patterns. March 2008.